

Visualizing uncertainty

+ ...

Maithreyi Gopalan
Week 9

Agenda

- Common ways of visualizing uncertainty
 - And how to implement them with `{ggplot2}`
- Framing uncertainty as relative frequencies
 - Discrete probabilities
 - Non-discrete probabilities
- Understanding standard errors
 - Non-standard ways of visualizing SEs
- HOPs (pretty quick)
 - Also bootstrapping

Agenda (continued)

Assuming we still have time (I think we will), we will also at least introduce **survey weights**. We probably will not have time for Shiny App deployments, but I have slides for them as well in case we do.

A quick warning

This will be a little more stats focused than basically any lecture we have had in this course so far

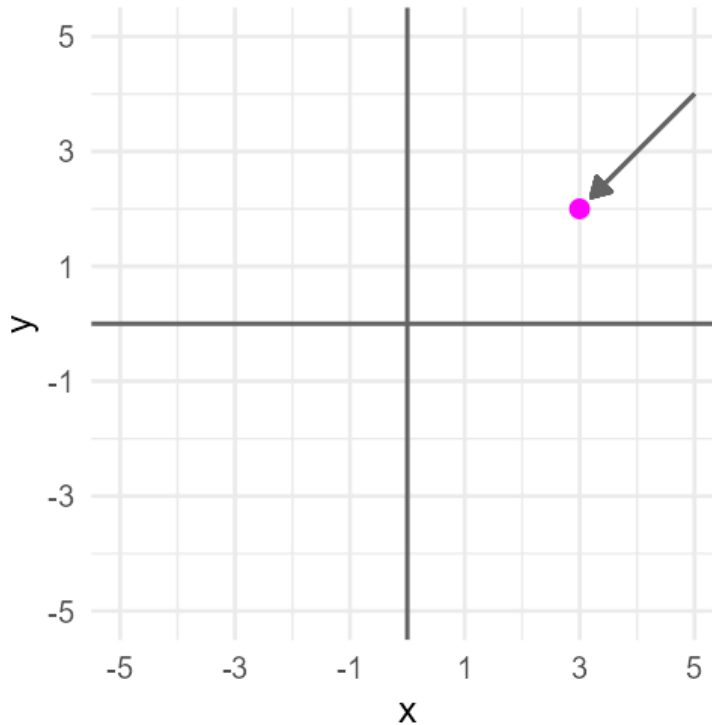
Uncertainty

Learning objectives

1. Understand there are lots of different ways to visualize uncertainty, and the best method may often be non-standard.
2. Understand how to implement basic methods, and the resources available to you to implement more advanced methods

The primary problem

- When we see a point on a plot, we interpret it as **THE** value.



Let's have Dr. Kay explain

Matthew Kay Keynote at Tapestry 2018: A biased tour of th...

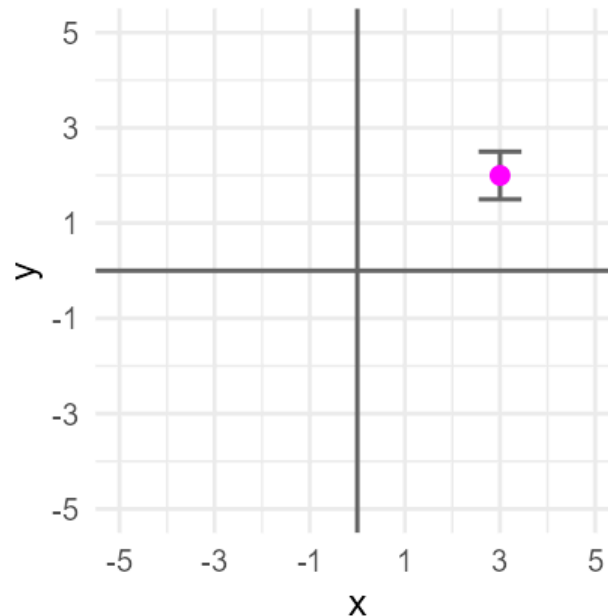


Some secondary problems

- We're not great at understanding probabilities
- We regularly round probabilities to 100% or 0%
- As probabilities move to the tails, we're generally worse

How do we typically communicate uncertainty?

Error bars



How?

Vertical error bars

`geom_errorbar`

- Requires `ymín` and `ymax` aesthetics
- You have to supply these - no calculation for you

Horizontal error bars

`geom_errorbarh`

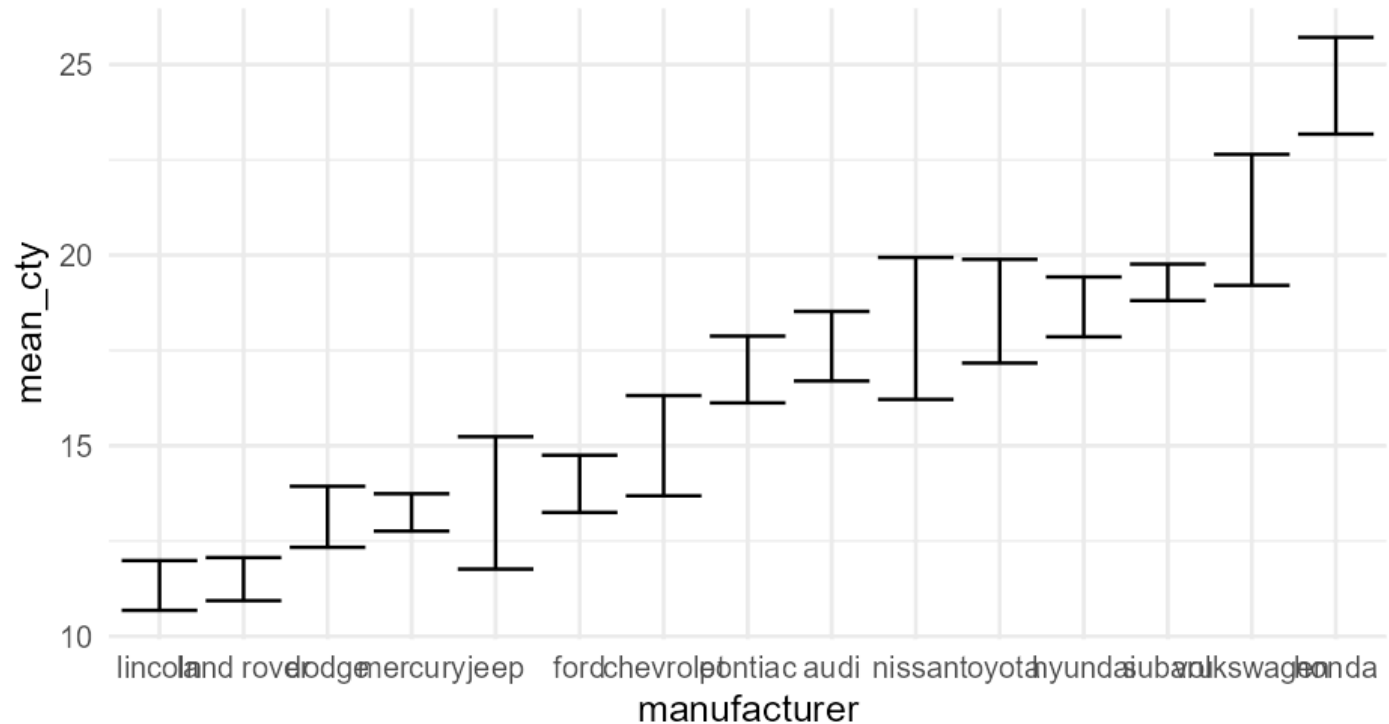
- Requires `xmín` and `xmax`

Example

```
mpg_by_man <- mpg %>%  
  group_by(manufacturer) %>%  
  summarize(mean_cty = mean(cty),  
            se_cty = sd(cty) / sqrt(n()))  
  
head(mpg_by_man)
```

```
## # A tibble: 6 × 3  
##   manufacturer mean_cty se_cty  
##   <chr>          <dbl> <dbl>  
## 1 audi           17.6  0.465  
## 2 chevrolet       15    0.671  
## 3 dodge          13.1  0.409  
## 4 ford           14    0.383  
## 5 honda          24.4  0.648  
## 6 hyundai        18.6  0.401
```

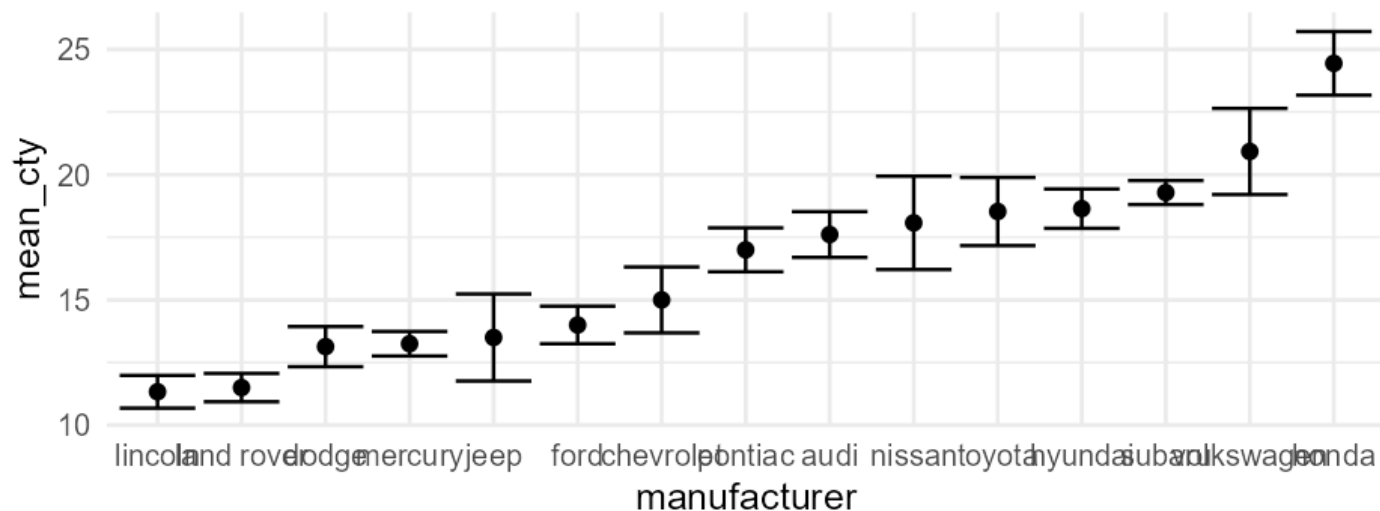
```
mpg_by_man %>%
  mutate(manufacturer = fct_reorder(manufacturer, mean_cty)) %>%
  ggplot(aes(manufacturer, mean_cty)) +
  geom_errorbar(
    aes(ymin = mean_cty + qnorm(0.025) * se_cty,
        ymax = mean_cty + qnorm(0.975) * se_cty)
  )
```



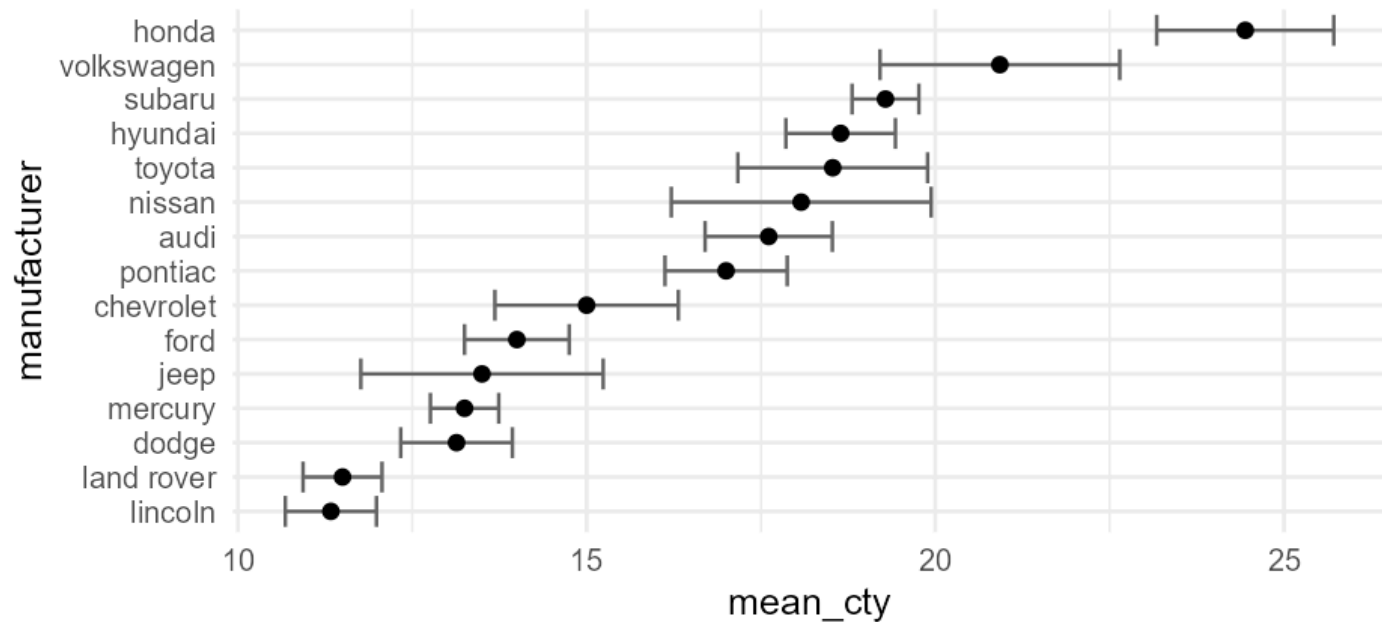
Put points on top

Not under

```
mpg_by_man %>%  
  mutate(manufacturer = fct_reorder(manufacturer, mean_cty)) %>%  
  ggplot(aes(manufacturer, mean_cty)) +  
    geom_errorbar(  
      aes(ymin = mean_cty + qnorm(0.025) * se_cty,  
          ymax = mean_cty + qnorm(0.975) * se_cty)  
    ) +  
    geom_point()
```



```
mpg_by_man %>%
  mutate(manufacturer = fct_reorder(manufacturer, mean_cty)) %>%
  ggplot(aes(mean_cty, manufacturer)) +
    geom_errorbarh(
      aes(xmin = mean_cty - 1.96 * se_cty,
          xmax = mean_cty + 1.96 * se_cty),
      color = "gray40"
    ) +
    geom_point()
```



Quick Example

- Let's use the Palmer penguins dataset
- Plot the mean `bill_length_mm` for each species with 95% confidence intervals

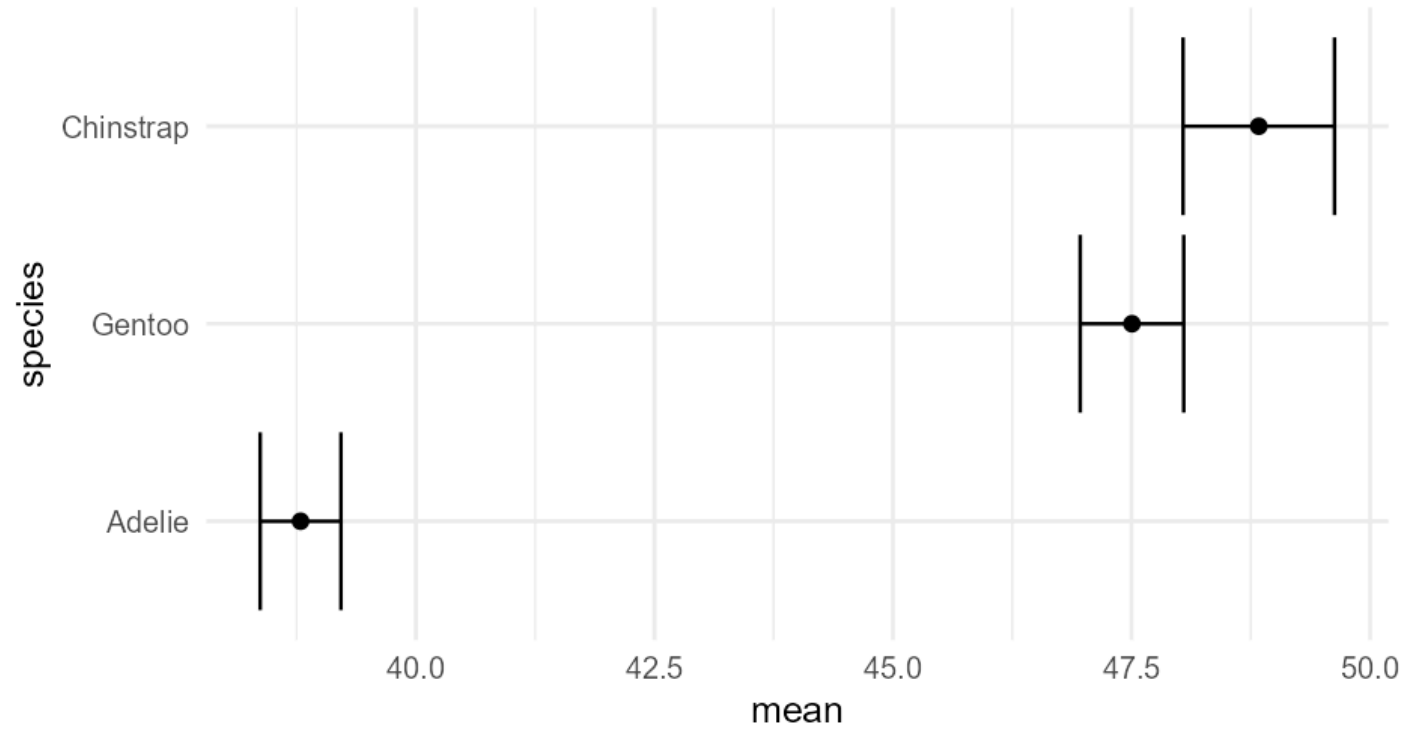
```
library(palmerpenguins)
penguins
```

```
## # A tibble: 344 × 8
##   species island   bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
##   <fct>   <fct>         <dbl>         <dbl>         <int>         <int>
## 1 Adelie  Torgersen         39.1          18.7          181          3750
## 2 Adelie  Torgersen         39.5          17.4          186          3800
## 3 Adelie  Torgersen         40.3          18           195          3250
## 4 Adelie  Torgersen         NA           NA           NA           NA
## 5 Adelie  Torgersen         36.7          19.3          193          3450
## 6 Adelie  Torgersen         39.3          20.6          190          3650
## 7 Adelie  Torgersen         38.9          17.8          181          3625
## 8 Adelie  Torgersen         39.2          19.6          195          4675
## 9 Adelie  Torgersen         34.1          18.1          193          3475
## 10 Adelie Torgersen         42           20.2          190          4250
## #> 334 more rows
```


Solution

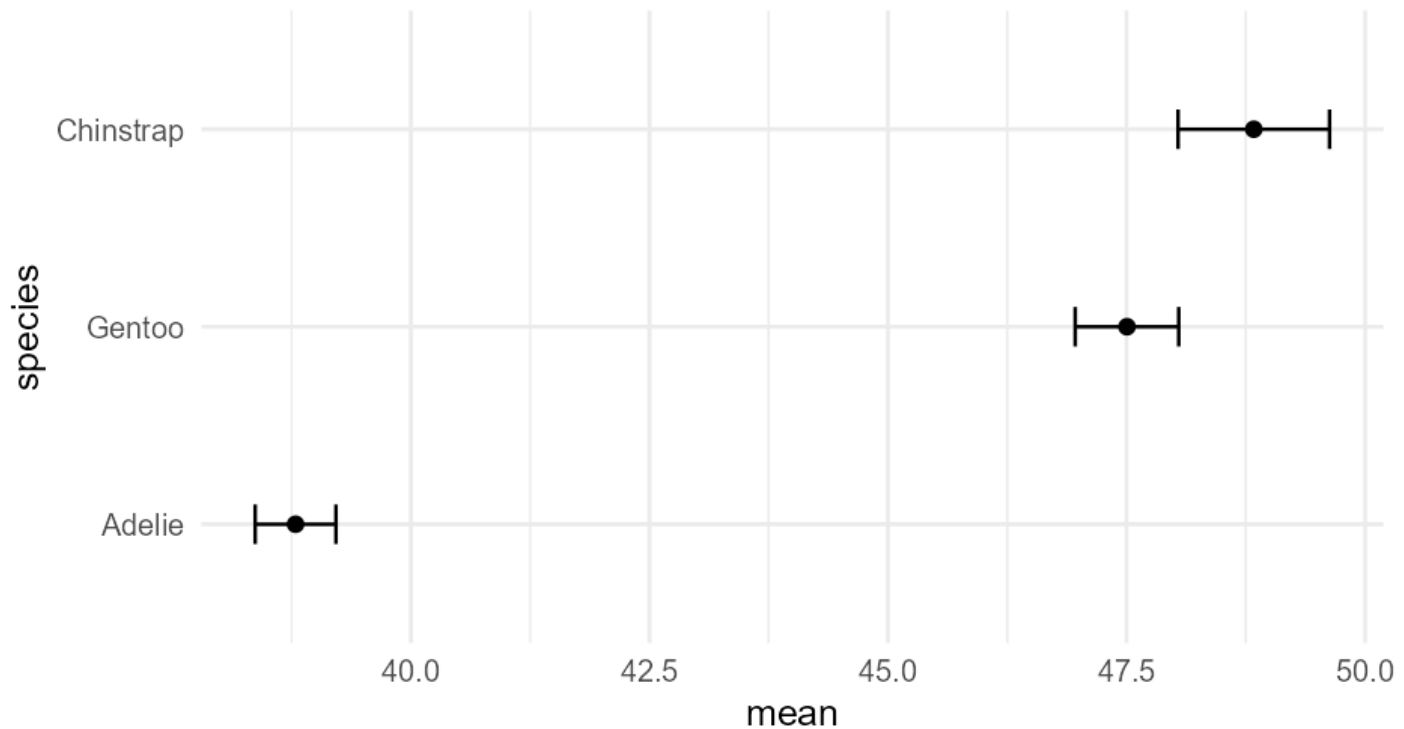
Or one possible solution

```
mn_se <- penguins %>%  
  group_by(species) %>%  
  summarize(  
    mean = mean(bill_length_mm, na.rm = TRUE),  
    se = sd(bill_length_mm, na.rm = TRUE) / sqrt(n()),  
    lower = mean + qnorm(0.025) * se,  
    upper = mean + qnorm(0.975) * se,  
  ) %>%  
  mutate(species = fct_reorder(species, mean))  
  
ggplot(mn_se, aes(mean, species)) +  
  geom_errorbarh(aes(xmin = lower, xmax = upper)) +  
  geom_point()
```



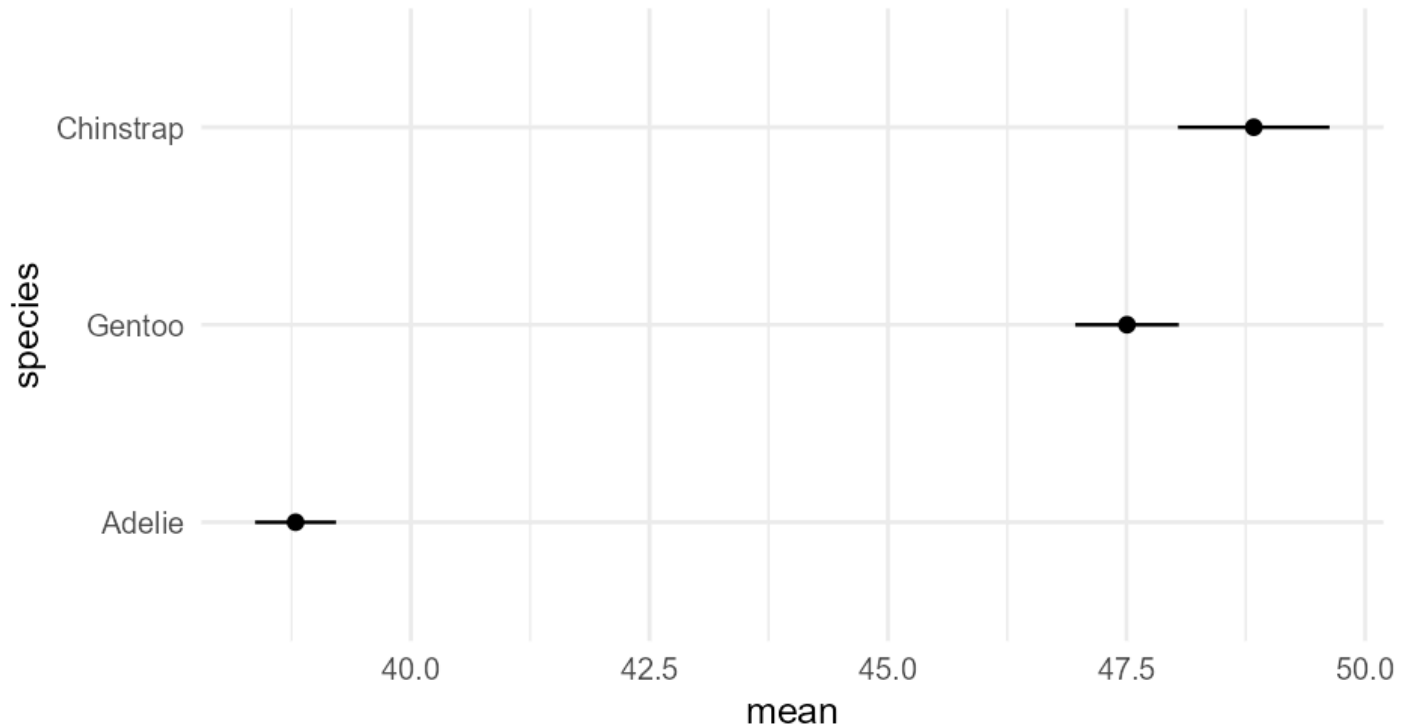
Change the height

```
ggplot(mn_se, aes(mean, species)) +  
  geom_errorbarh(aes(xmin = lower, xmax = upper), height = 0.2) +  
  geom_point()
```



Slight variant

```
ggplot(mn_se, aes(mean, species)) +  
  geom_linerange(aes(xmin = lower, xmax = upper)) +  
  geom_point()
```



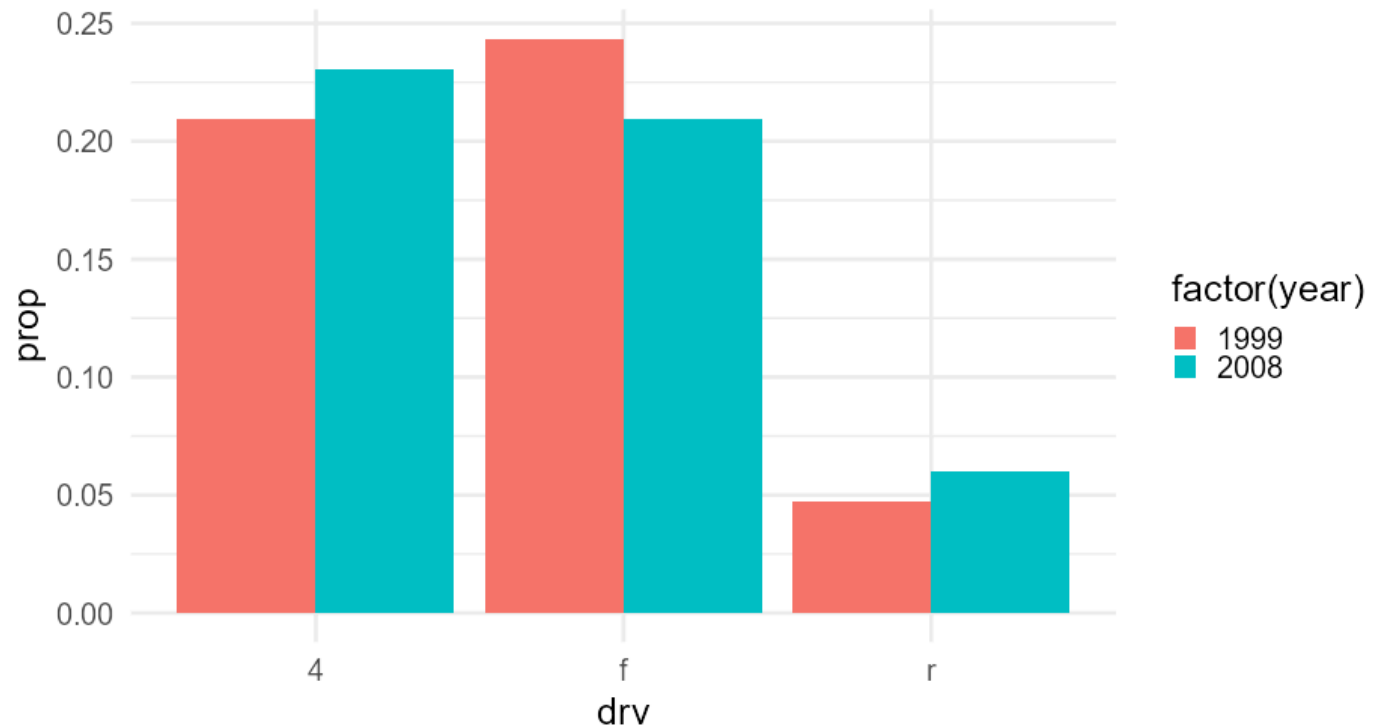
Dodging

```
props <- mpg %>%  
  count(drv, year) %>%  
  mutate(prop = n / sum(n),  
         prop_se = sqrt((prop * (1 - prop)) / n))  
  
head(props)
```

```
## # A tibble: 6 × 5  
##   drv    year     n  prop prop_se  
##   <chr> <int> <int> <dbl>  <dbl>  
## 1 4      1999     49 0.209  0.0581  
## 2 4      2008     54 0.231  0.0573  
## 3 f      1999     57 0.244  0.0569  
## 4 f      2008     49 0.209  0.0581  
## 5 r      1999     11 0.0470 0.0638  
## 6 r      2008     14 0.0598 0.0634
```

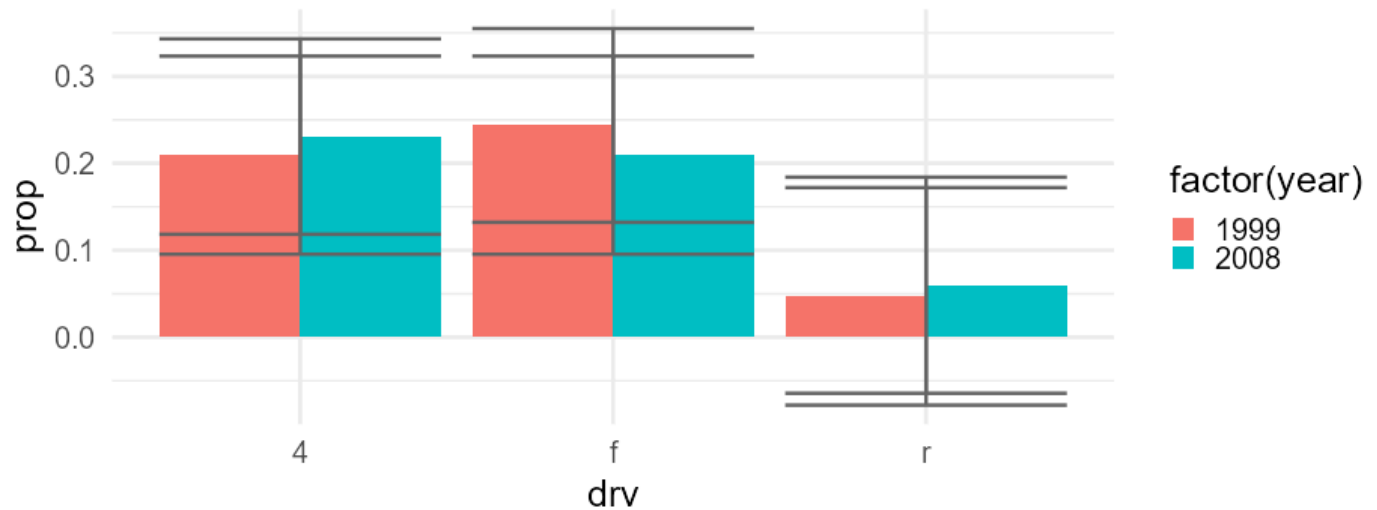
Bar plot

```
ggplot(props, aes(drv, prop)) +  
  geom_col(aes(fill = factor(year)), position = "dodge")
```





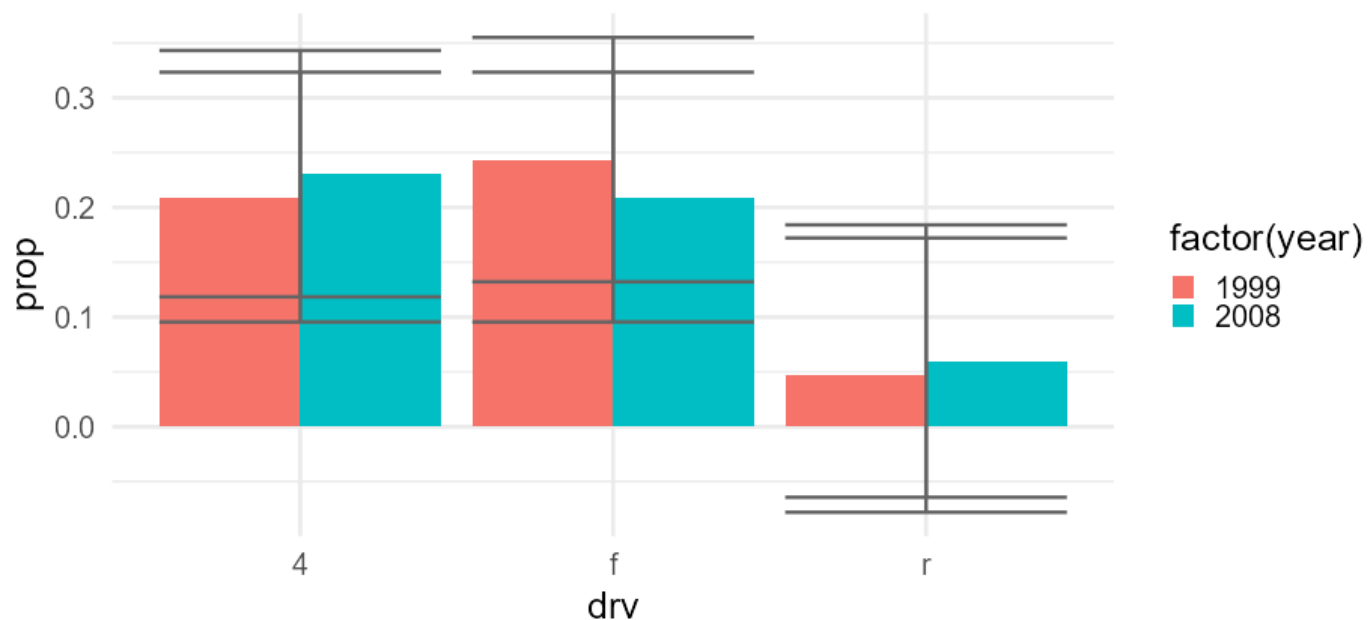
```
ggplot(props, aes(drv, prop)) +  
  geom_col(aes(fill = factor(year)), position = "dodge") +  
  geom_errorbar(  
    aes(ymin = prop - 1.96 * prop_se,  
        ymax = prop + 1.96 * prop_se),  
    color = "gray40"  
  )
```



```

pd <- position_dodge(.9)
ggplot(props, aes(drv, prop)) +
  geom_col(aes(fill = factor(year)), position = pd) +
  geom_errorbar(
    aes(ymin = prop - 1.96 * prop_se,
        ymax = prop + 1.96 * prop_se),
    color = "gray40",
    position = pd
  )

```

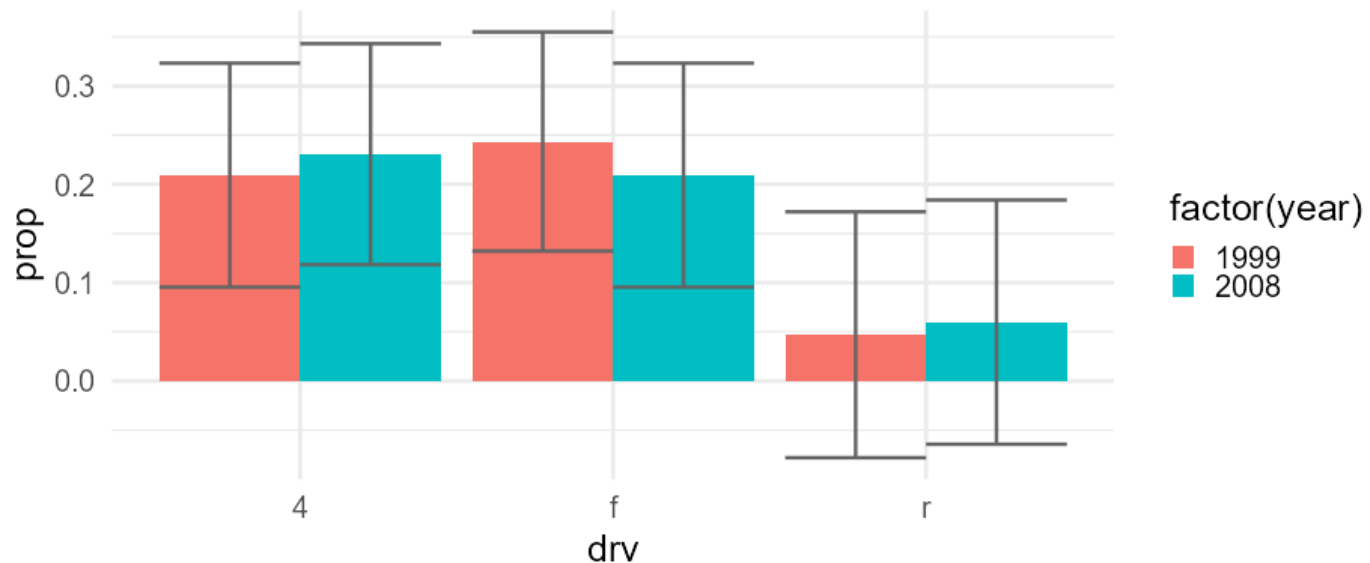




```

pd <- position_dodge(.9)
ggplot(props, aes(drv, prop)) +
  geom_col(aes(fill = factor(year)), position = pd) +
  geom_errorbar(
    aes(ymin = prop - 1.96 * prop_se,
        ymax = prop + 1.96 * prop_se,
        group = year
    ),
    color = "gray40",
    position = pd
  )

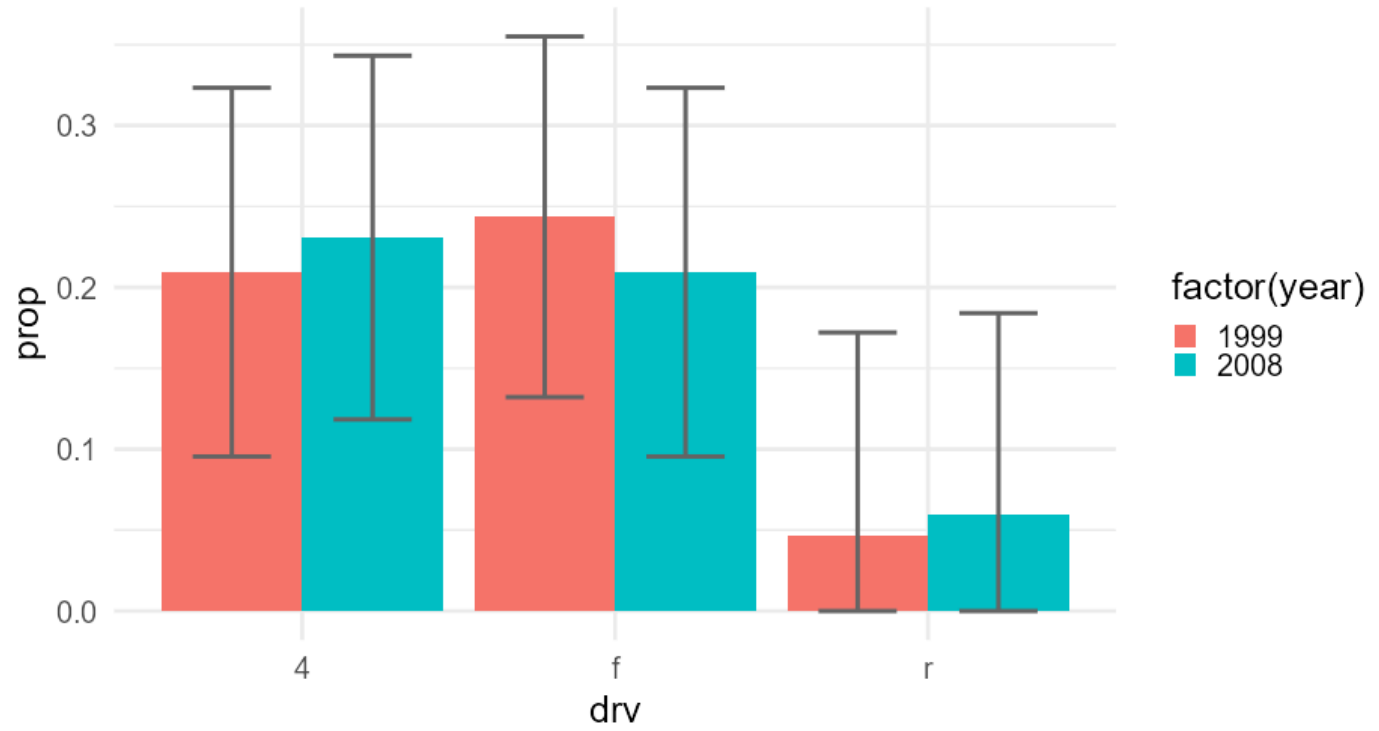
```



```

pd <- position_dodge(.9)
ggplot(props, aes(drv, prop)) +
  geom_col(aes(fill = factor(year)), position = pd) +
  geom_errorbar(
    aes(
      ymin = ifelse(
        prop - 1.96 * prop_se < 0,
        0,
        prop - 1.96*prop_se
      ),
      ymax = prop + 1.96 * prop_se,
      group = year
    ),
    color = "gray40",
    position = pd,
    width = 0.5,
    size = 1.4
  )

```



Explain error bars

Error bars could represent any of the following

- Standard deviation (of the data)
- Standard error (of the estimate)
- Confidence interval (of the estimate)

Possibly through a caption, make sure your audience knows what your represent.

If Confidence intervals, state the interval size (e.g., 68%, 90%, 95%)

Thinking about uncertainty

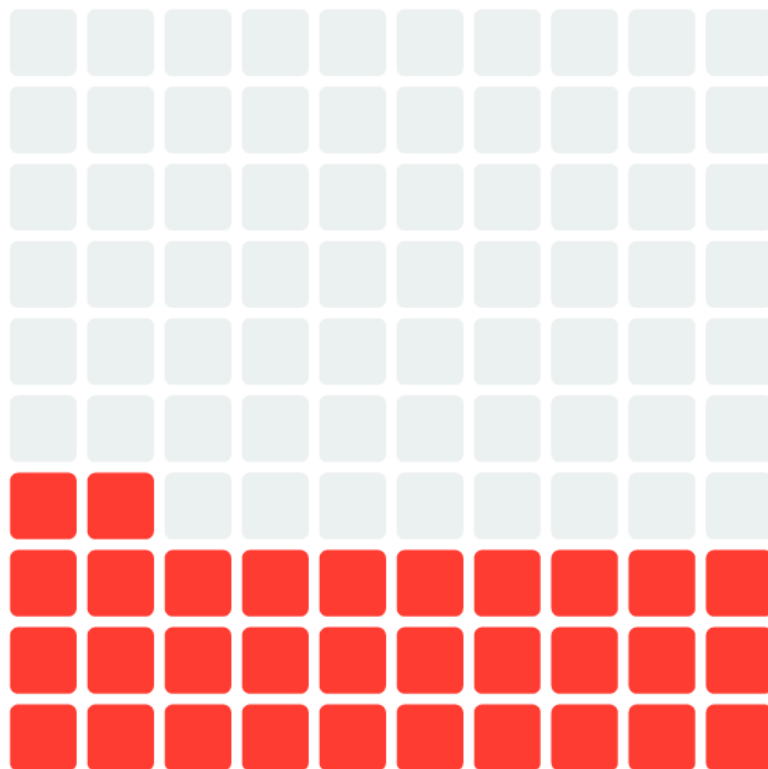
Uncertainty means exactly what it sounds like - we are not 100% sure.

- We are nearly always uncertain of future events (forecasting)
- We can also be uncertain about past events
 - I saw a parked car at 8 AM, but the next time I looked at 2PM it was gone. What time did it leave?

Quantifying uncertainty

- We quantify our uncertainty mathematically using probability
- Framing probabilities as frequencies is generally more intuitive

Framing a single uncertainty



32%

How do we make these?

- Start out by making a grid

```
grid <- expand.grid(x = 1:20, y = 1:20)
head(grid)
```

```
##    x y
##  1 1 1
##  2 2 1
##  3 3 1
##  4 4 1
##  5 5 1
##  6 6 1
```

```
tail(grid)
```

```
##      x  y
## 395 15 20
## 396 16 20
## 397 17 20
## 398 18 20
## 399 19 20
## 400 20 20
```

Look at the grid

```
ggplot(grid, aes(x, y)) +  
  geom_tile(color = "gray40",  
            fill = "white") +  
  theme_void()
```



Create occurrence rate

- For each sequence of x , create a variable that has the given occurrence rate

How?

- Plenty of options, here's one

Consider 10%

```
# create your grid
grid <- expand.grid(x = 1:20, y = 1:20)

# n to sample
n_sample <- nrow(grid) * .10

set.seed(86753098)
samp <- sample(seq_len(nrow(grid)), n_sample)
head(samp)
```

```
## [1] 318 134 180 283 177 248
```

```
length(samp)
```

```
## [1] 40
```

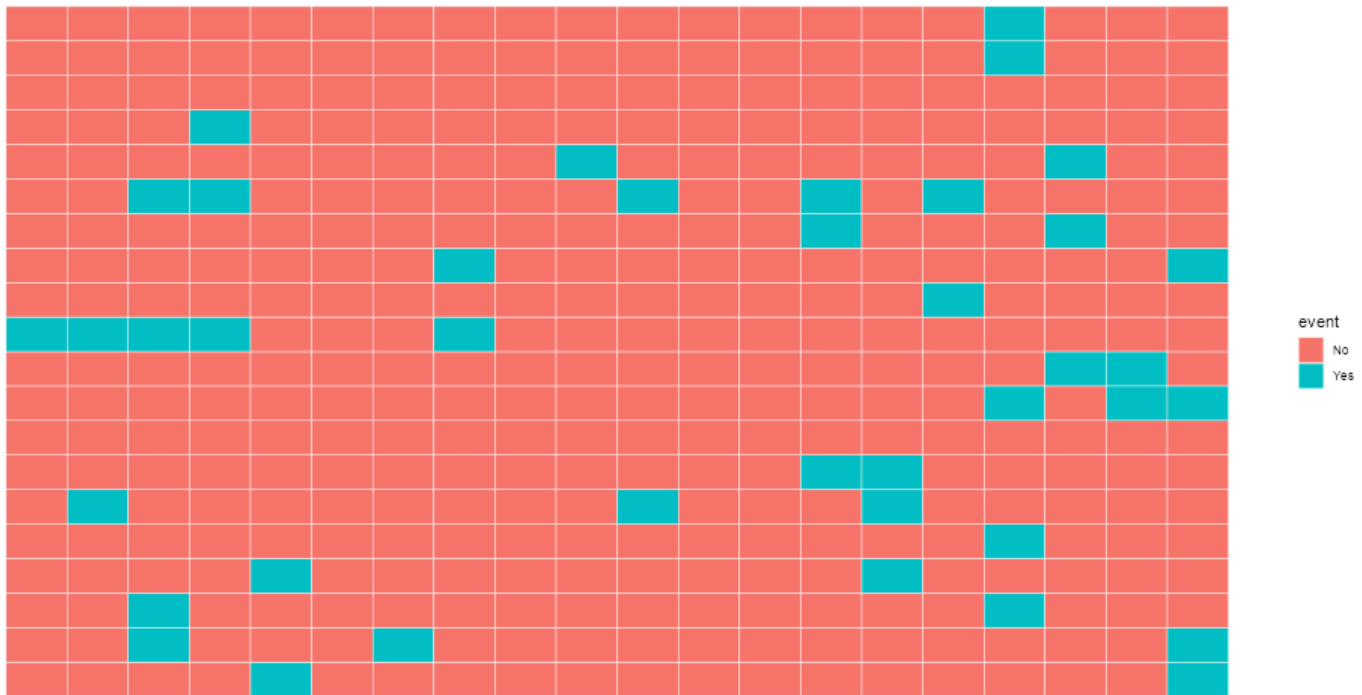
Create the variable

```
grid <- grid %>%  
  rownames_to_column("row_id") %>%  
  mutate(event = ifelse(row_id %in% samp, "Yes", "No"))  
head(grid)
```

```
##   row_id x y event  
## 1      1 1 1   No  
## 2      2 2 1   No  
## 3      3 3 1   No  
## 4      4 4 1   No  
## 5      5 5 1  Yes  
## 6      6 6 1   No
```

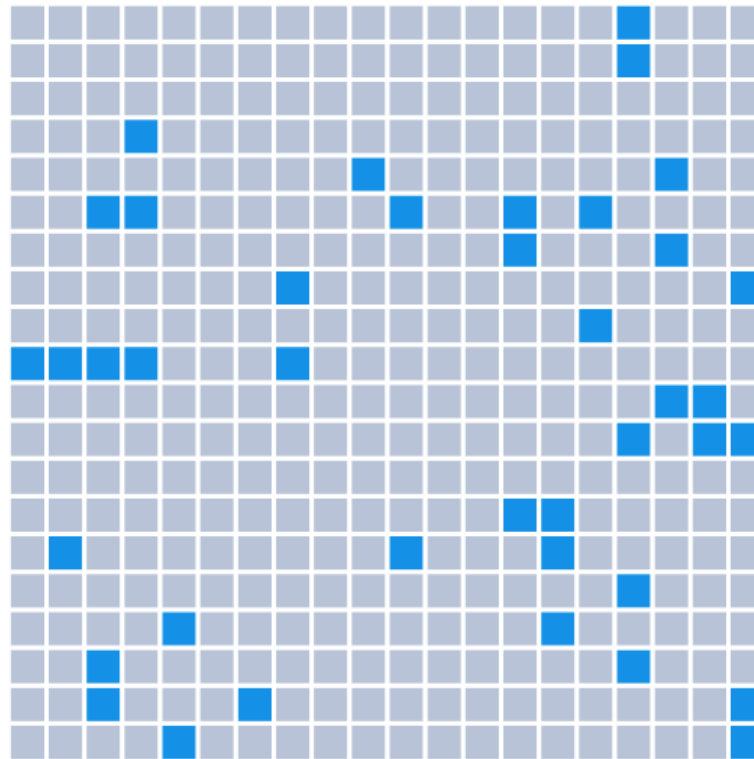
Fill in

```
ggplot(grid, aes(x, y)) +  
  geom_tile(aes(fill = event), color = "white") +  
  theme_void()
```



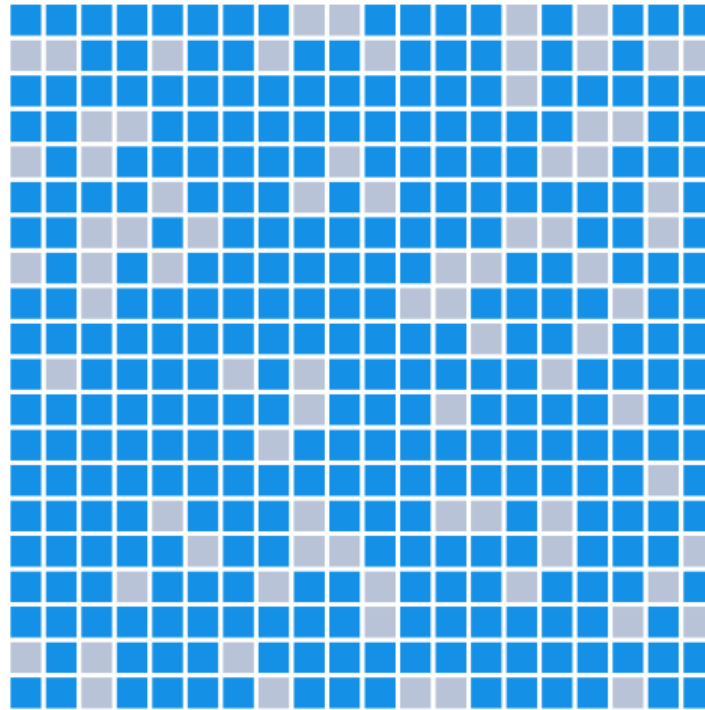
Customize

```
library(colorspace)
ggplot(grid, aes(x, y)) +
  geom_tile(aes(fill = event), color = "white", size = 1.4) +
  scale_fill_manual(
    name = "Event Occurred",
    values = c(
      desaturate(lighten("#1694E8", 0.5), 0.7),
      "#1694E8"
    )
  ) +
  coord_fixed() +
  theme_void() +
  theme(legend.position = c(0.75, 0),
        legend.direction = "horizontal",
        plot.margin = margin(b = 1, unit = "cm"))
```



Event Occurred No Yes

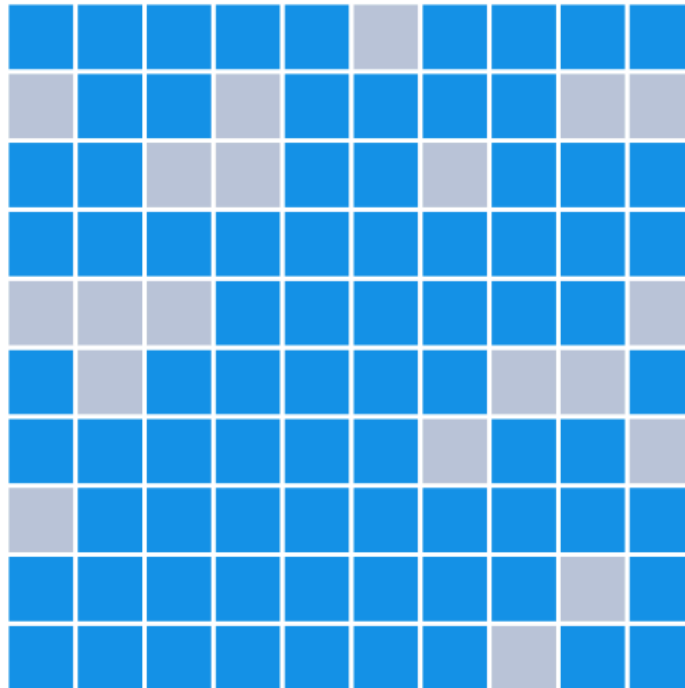
Chance of rain



Event Occurred ■ No ■ Yes

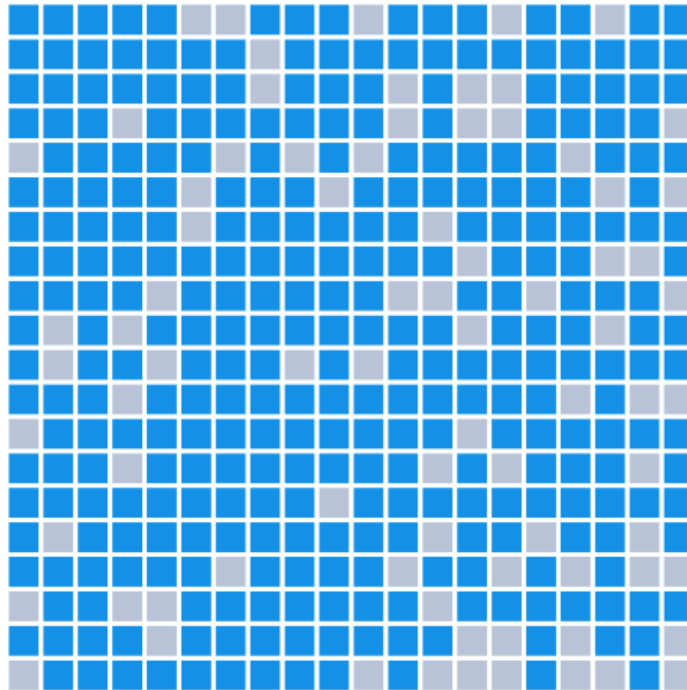
Vary grid size

10 x 10



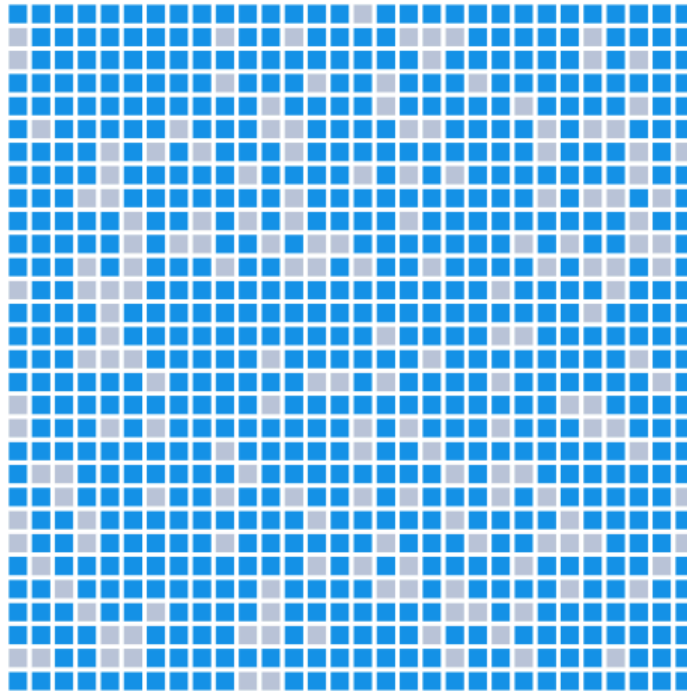
Vary grid size

20 x 20



Vary grid size

30 x 30



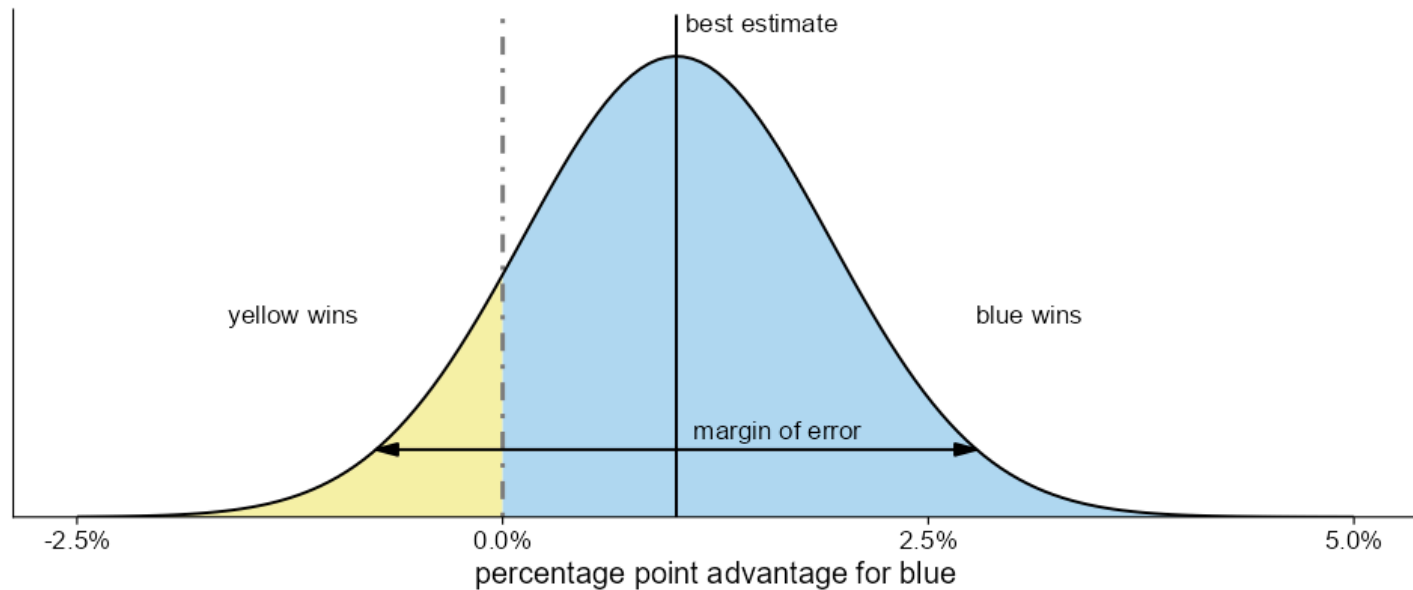
Optimal size might depend on your venue

Non- discrete probabilities

Hypothetical

Blue party has 1% advantage w/ margin of error of 1.76 percentage points

Who will win and by how much?



A bit of math

Our distribution was defined by $\mu = 1.02$ and $sd = 0.9$.

- What's the chance the end result is **below zero**? (yellow wins)

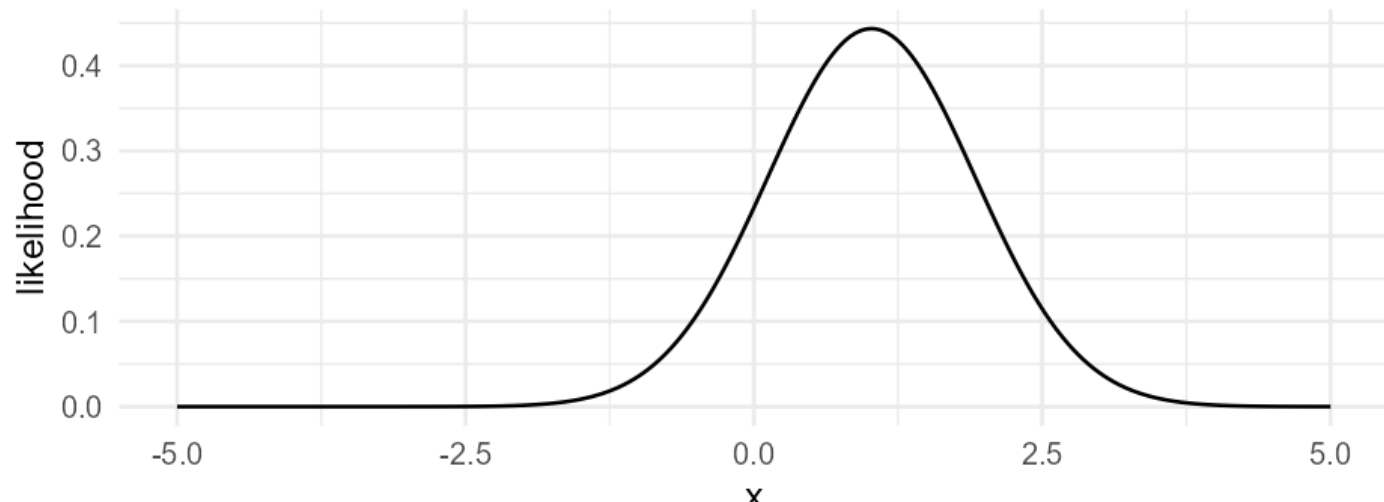
The hard way

Calculate the exact probability

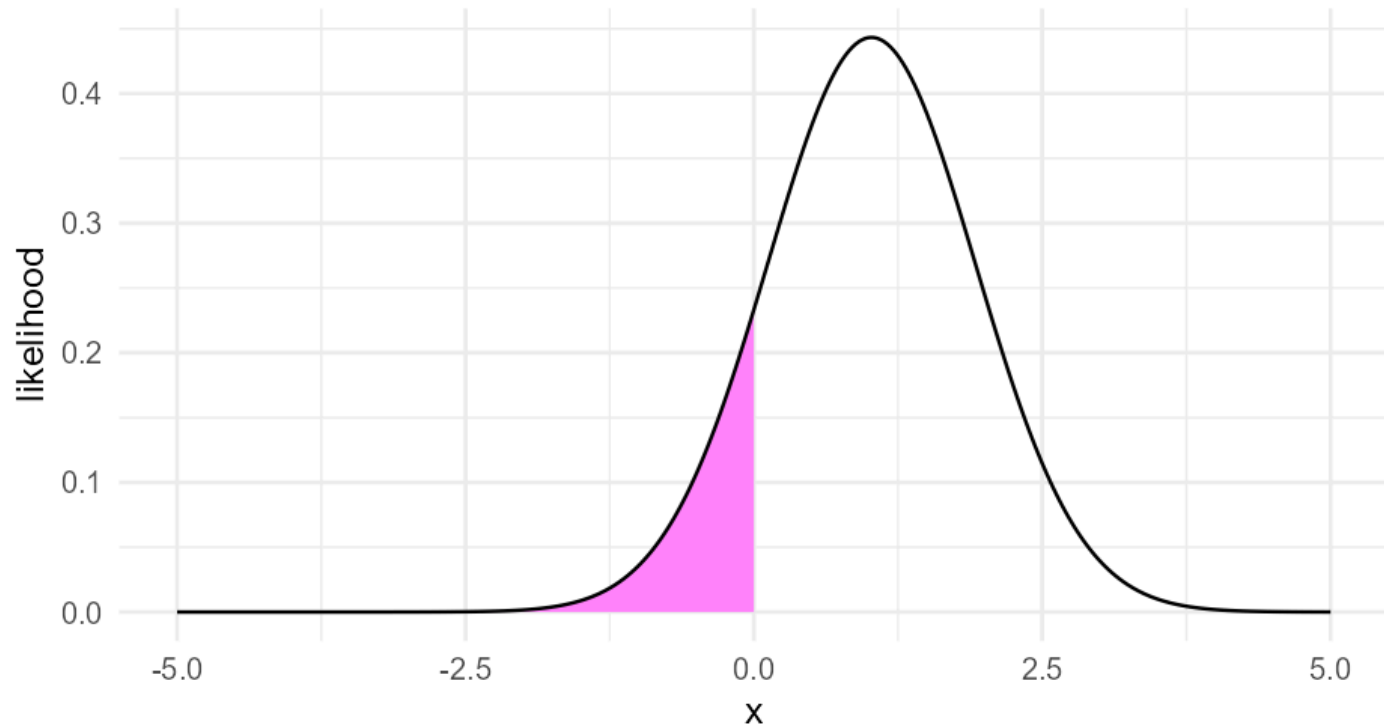
The distribution

```
x <- seq(-5, 5, 0.001) # some sample data
likelihood <- dnorm(x, 1.02, 0.9) # probability of occurring
sim <- data.frame(x, likelihood)

ggplot(sim, aes(x, likelihood)) +
  geom_line(size = 1.2)
```



How do we calculate this portion?



Integrate

```
zab <- filter(sim, x <= 0)  
pracma::trapz(zab$x, zab$likelihood)
```

```
## [1] 0.1285372
```

Easier: Simulate

```
random_draws <- rnorm(1e5, 1.02, 0.9)
table(random_draws > 0) / 1e5
```

```
##
##  FALSE    TRUE
## 0.12968 0.87032
```

Problem

This approach works okay, but...

- We're not great at interpreting probabilities
- Not great at inferring a probability from a density

Discretized plot

```
ppoints(50)
```

```
## [1] 0.01 0.03 0.05 0.07 0.09 0.11 0.13 0.15 0.17 0.19 0.21 0.23 0.25 0.27 0.29  
## [32] 0.63 0.65 0.67 0.69 0.71 0.73 0.75 0.77 0.79 0.81 0.83 0.85 0.87 0.89 0.91
```

```
qnorm(ppoints(50), 1.02, 0.9)
```

```
## [1] -1.073713087 -0.672714247 -0.460368264 -0.308211925 -0.186679530 -0.083875  
## [13] 0.412959225 0.468468308 0.521953752 0.573734687 0.624078151 0.673211  
## [25] 0.997437983 1.042562017 1.087742876 1.133095212 1.178736748 1.224790  
## [37] 1.571531692 1.627040775 1.684962164 1.745779122 1.810106666 1.878748  
## [49] 2.712714247 3.113713087
```

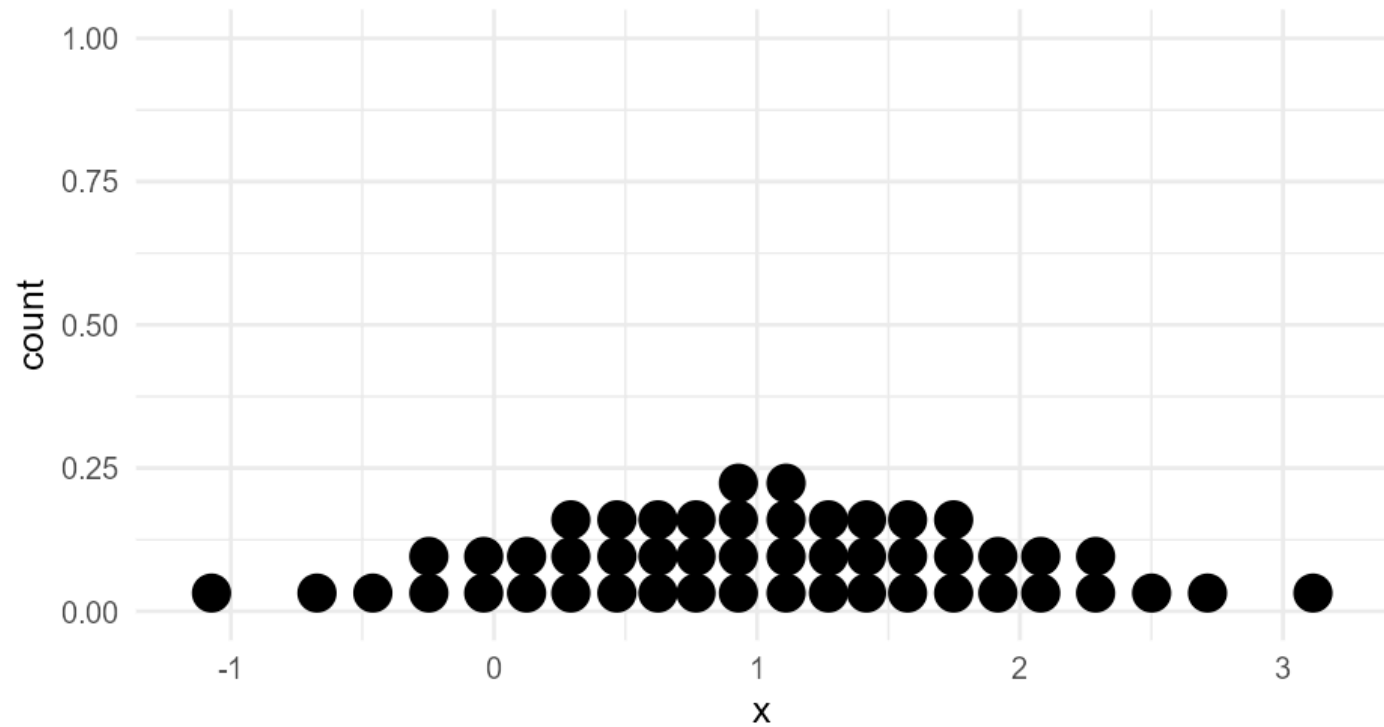
```
discretized <- data.frame(  
  x = qnorm(ppoints(50), 1.02, 0.9)  
  ) %>%  
  mutate(winner = ifelse(x <= 0, "#b1daf4", "#f8f1a9"))  
  
head(discretized)
```

```
##           x  winner  
## 1 -1.07371309 #b1daf4  
## 2 -0.67271425 #b1daf4  
## 3 -0.46036826 #b1daf4  
## 4 -0.30821193 #b1daf4  
## 5 -0.18667953 #b1daf4  
## 6 -0.08387531 #b1daf4
```

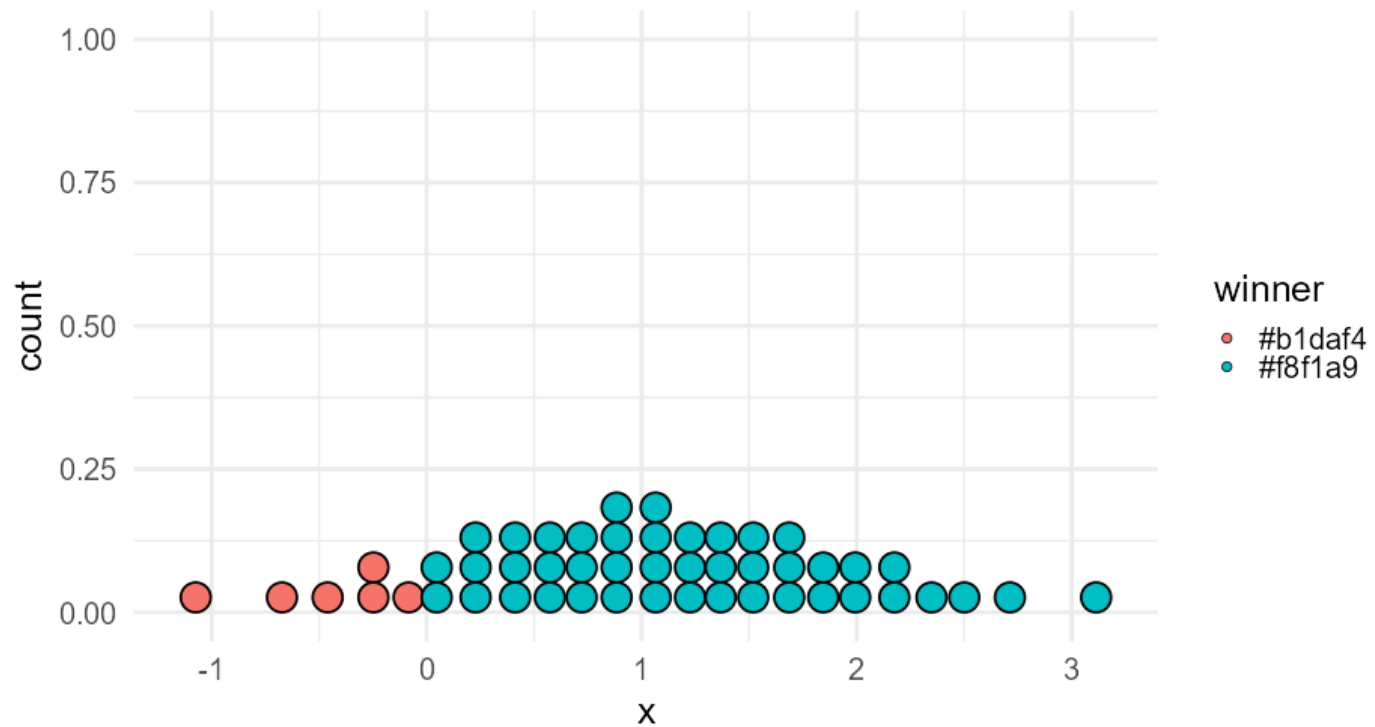
```
tail(discretized)
```

```
##           x  winner  
## 45  2.123875 #f8f1a9  
## 46  2.226680 #f8f1a9  
## 47  2.348212 #f8f1a9  
## 48  2.500368 #f8f1a9  
## 49  2.712714 #f8f1a9  
## 50  3.113713 #f8f1a9
```

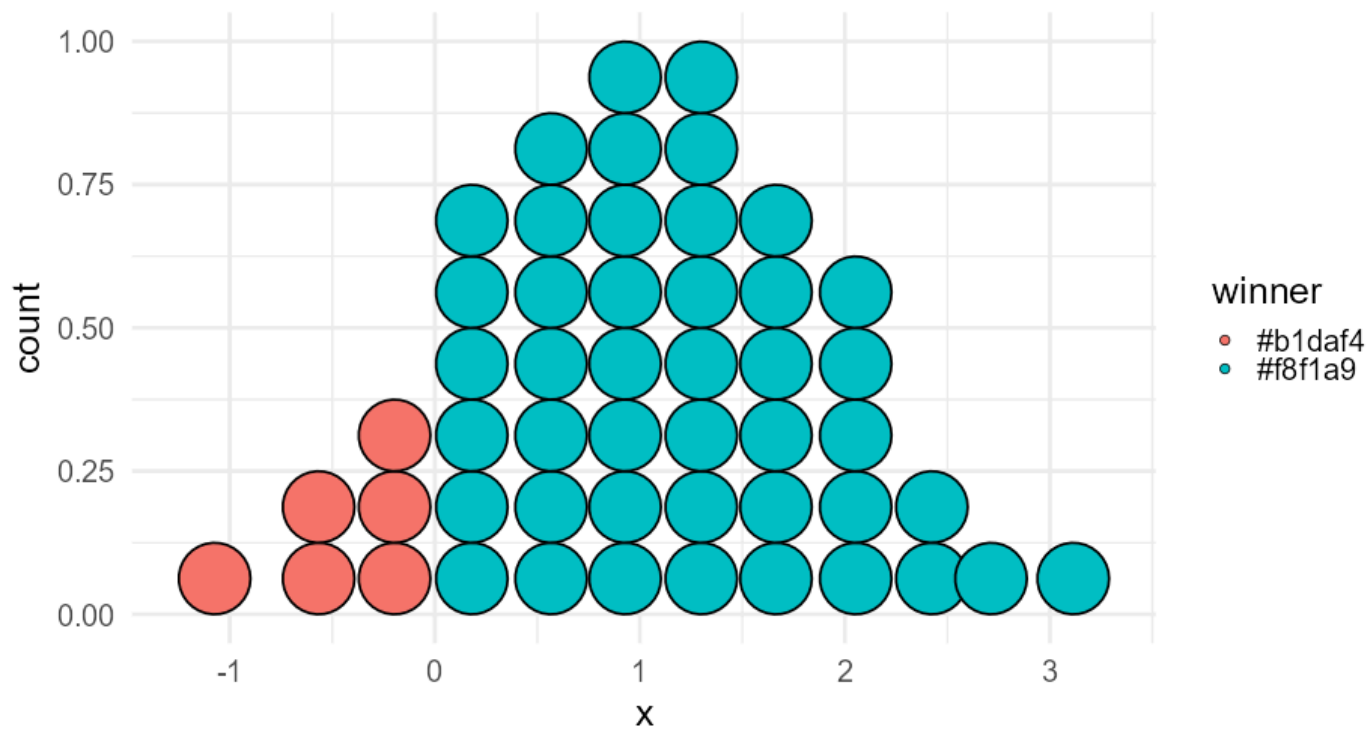
```
ggplot(discretized, aes(x)) +  
  geom_dotplot()
```



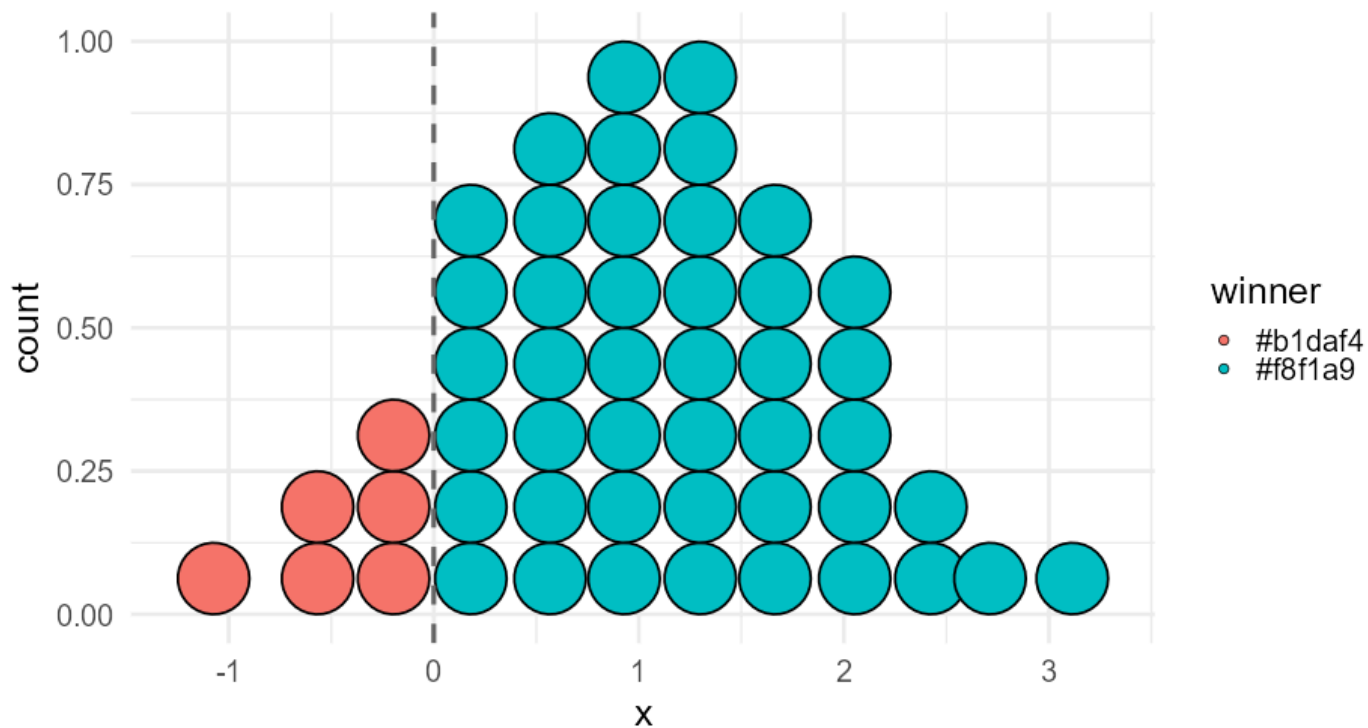
```
ggplot(discretized, aes(x)) +  
  geom_dotplot(aes(fill = winner))
```



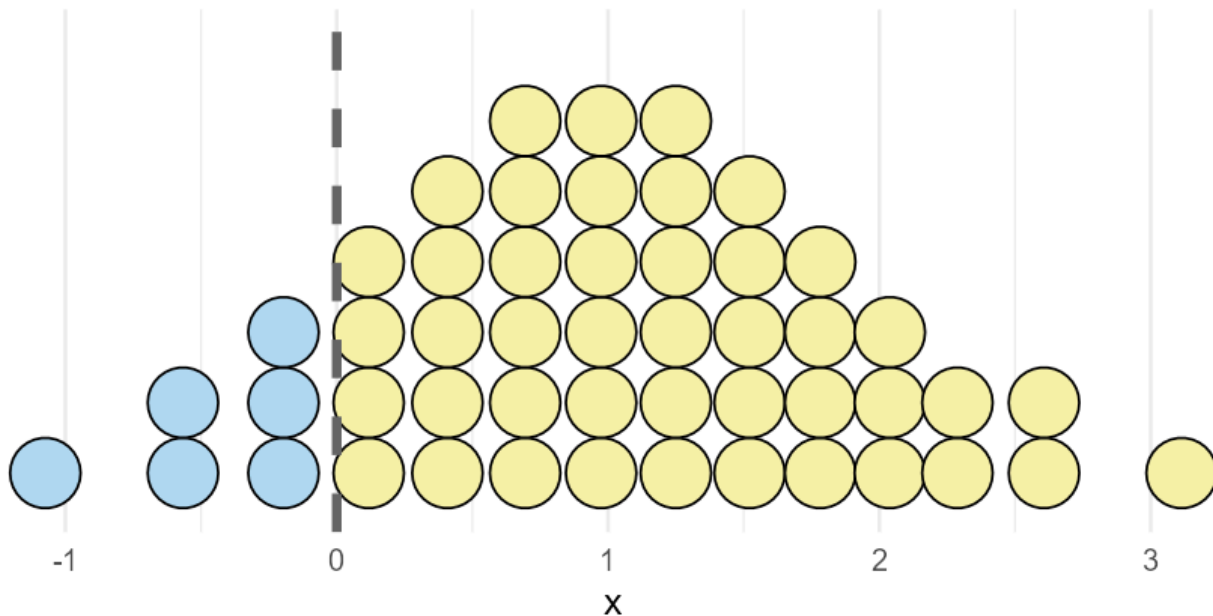

```
ggplot(discretized, aes(x)) +  
  geom_dotplot(aes(fill = winner), binwidth = 0.35)
```



```
ggplot(discretized, aes(x)) +
  geom_dotplot(aes(fill = winner), binwidth = 0.35) +
  geom_vline(xintercept = 0,
             color = "gray40",
             linetype = "dashed",
             size = 1.5)
```

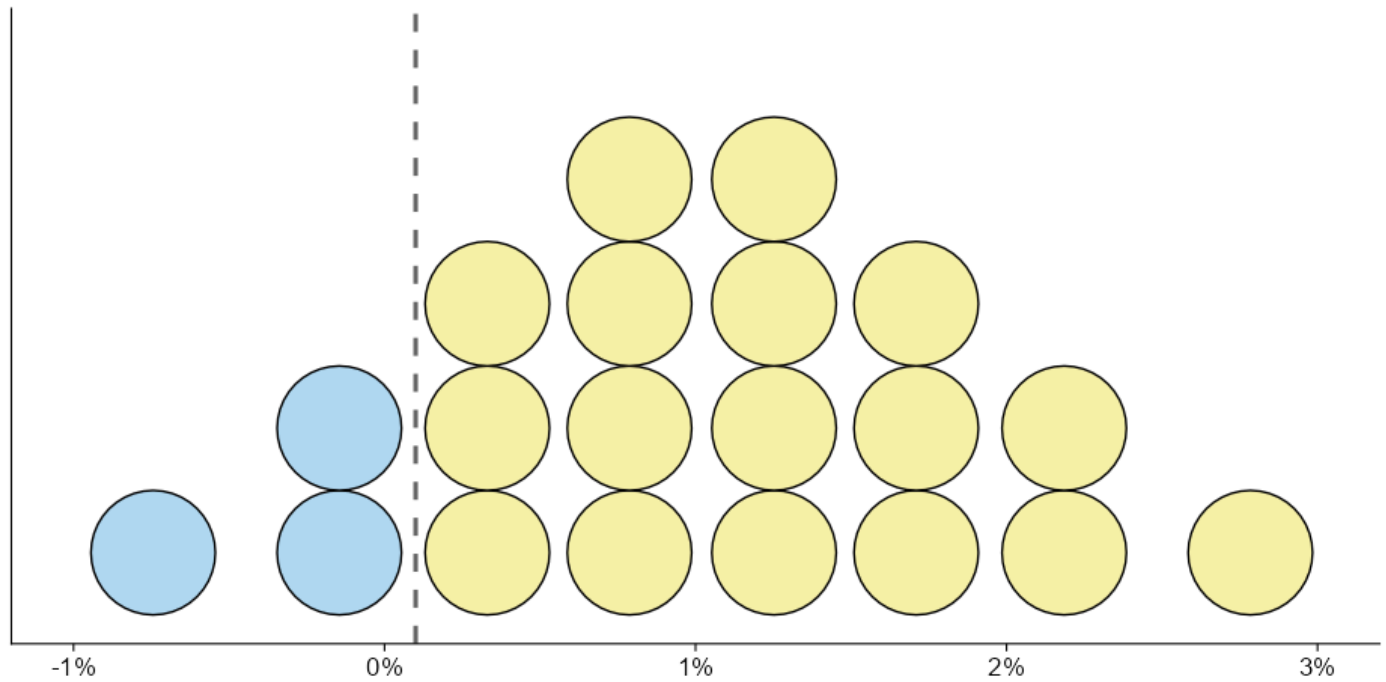


```
ggplot(discretized, aes(x)) +
  geom_dotplot(aes(fill = winner), binwidth = 0.26) +
  geom_vline(xintercept = 0,
             color = "gray40",
             linetype = 2,
             size = 3) +
  scale_fill_identity(guide = "none") +
  scale_y_continuous(name = "",
                     breaks = NULL)
```



Probs too many though

```
discretized2 <- data.frame(  
  x = qnorm(ppoints(20), 1.02, 0.9)  
  ) %>%  
  mutate(winner = ifelse(x <= 0, "#b1daf4", "#f8f1a9"))  
  
ggplot(discretized2, aes(x)) +  
  geom_dotplot(aes(fill = winner), binwidth = 0.4) +  
  geom_vline(  
    xintercept = 0.1,  
    color = "gray40",  
    linetype = 2,  
    size = 1.4) +  
  scale_fill_identity(guide = "none") +  
  scale_x_continuous(  
    name = "",  
    limits = c(-1, 3),  
    labels = scales::percent_format(scale = 1)  
  ) +  
  theme_dviz_open(20, font_family = "Roboto Light") +  
  scale_y_continuous(breaks = NULL,  
                     name = "") +  
  labs(caption = "Each ball represents 5% probability")
```



Each ball represents 5% probability

Uncertainty of point estimates

Quick (hopefully) review

- What is a standard error?
- Standard deviation of the sampling distribution
- What is the sampling distribution?
- Samples from the underlying, population-based, generative distribution
- What does this mean, exactly?
- Let's simulate to explore

Simulation

- Imagine the "real" distribution has $\mu = 100$ and $\sigma = 10$.
- Let's draw a sample of 10 from this distribution

```
set.seed(123)
samp10a <- rnorm(n = 10, mean = 100, sd = 10)
samp10a
```

```
## [1] 94.39524 97.69823 115.58708 100.70508 101.29288 117.15065 104.60916 87.15065
```

- Calculate the mean

```
mean(samp10a)
```

```
## [1] 100.7463
```


Do it a second time

```
samp10b <- rnorm(n = 10, mean = 100, sd = 10)
samp10b
```

```
## [1] 112.24082 103.59814 104.00771 101.10683 94.44159 117.86913 104.97850 80.0
```

```
mean(samp10b)
```

```
## [1] 102.0862
```

Do it a bunch of times

```
samples <- replicate(1000, rnorm(10, mean = 100, sd = 10),  
                      simplify = FALSE)  
samples
```

```
## [[1]]  
## [1] 89.32176 97.82025 89.73996 92.71109 93.74961 83.13307 108.37787 101.13111 99.13111 99.13111  
##  
## [[2]]  
## [1] 104.26464 97.04929 108.95126 108.78133 108.21581 106.88640 105.53918 99.13111 99.13111 99.13111  
##  
## [[3]]  
## [1] 93.05293 97.92083 87.34604 121.68956 112.07962 88.76891 95.97115 95.97115 95.97115 95.97115  
##  
## [[4]]  
## [1] 102.53319 99.71453 99.57130 113.68602 97.74229 115.16471 84.51247 105.13111 105.13111 105.13111  
##  
## [[5]]  
## [1] 103.79639 94.97677 96.66793 89.81425 89.28209 103.03529 104.48210 100.13111 100.13111 100.13111  
##  
## [[6]]  
## [1] 95.08969 76.90831 110.05739 92.90799 93.11991 110.25571 97.15227 87.13111 87.13111 87.13111  
##  
## [[7]]  
## [1] 100.05764 103.85280 96.29340 106.44377 97.79513 103.31782 110.96839 104.13111 104.13111 104.13111  
##
```

Calculate all means

```
map_dbl(samples, mean) %>%  
  head()
```

```
## [1] 95.75441 103.22045 99.91284 102.21686 101.23084 96.37082
```

- What's the ***sd*** of these means? That's the standard error.

```
map_dbl(samples, mean) %>%  
  sd()
```

```
## [1] 3.144175
```

Sample size

Let's re-do this, pulling a sample of 100 each time.

```
samples2 <- replicate(1000, rnorm(100, mean = 100, sd = 10),  
                        simplify = FALSE)  
map_dbl(samples2, mean) %>%  
  sd()
```

```
## [1] 0.9728883
```

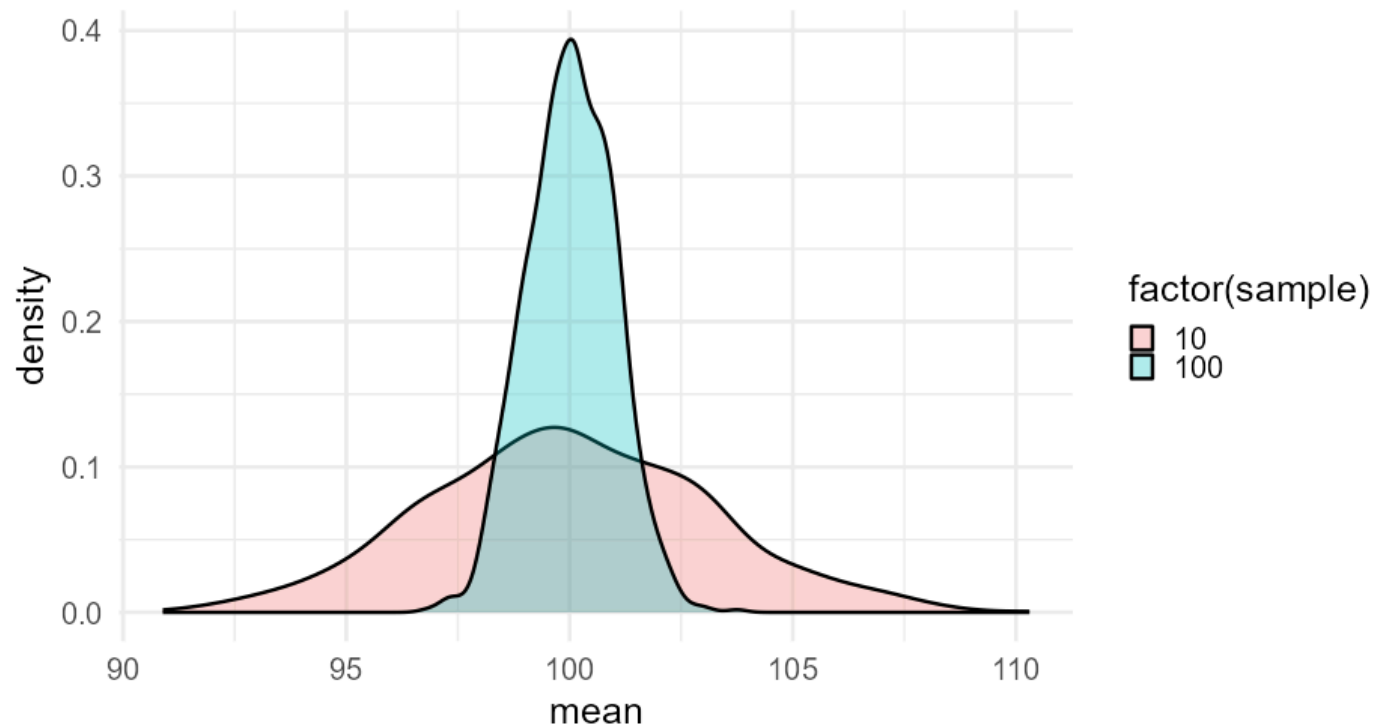
Visualize the sampling distributions

```
sample_means <- tibble(iter = rep(1:1000, 2),
                        sample = rep(c(10, 100), each = 1000),
                        mean = c(map_dbl(samples, mean),
                                map_dbl(samples2, mean))
                        )
```

```
sample_means
```

```
## # A tibble: 2,000 × 3
##   iter sample  mean
##   <int> <dbl> <dbl>
## 1     1     10  95.8
## 2     2     10 103.
## 3     3     10  99.9
## 4     4     10 102.
## 5     5     10 101.
## 6     6     10  96.4
## 7     7     10 103.
## 8     8     10 104.
## 9     9     10  96.0
## 10    10     10  96.8
## #> 1,990 more rows
```

```
ggplot(sample_means, aes(mean)) +  
  geom_density(aes(fill = factor(sample)), alpha = 0.3)
```



Fit a model

```
m <- lm(cty ~ displ + class, mpg)
summary(m)
```

```
##
## Call:
## lm(formula = cty ~ displ + class, data = mpg)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.2689 -1.1503 -0.0156  1.0341 12.9782
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    28.7768     1.4729   19.538  < 2e-16 ***
## displ         -2.1716     0.1747  -12.433  < 2e-16 ***
## classcompact  -3.5991     1.2522   -2.874  0.00444 **
## classmidsize  -3.6755     1.2063   -3.047  0.00259 **
## classminivan  -5.5951     1.3060   -4.284  2.71e-05 ***
## classpickup   -6.1825     1.1214   -5.513  9.60e-08 ***
## classsubcompact -2.6290     1.2369   -2.125  0.03464 *
## classsub      -5.5994     1.0872   -5.150  5.65e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

Visualize with standard errors

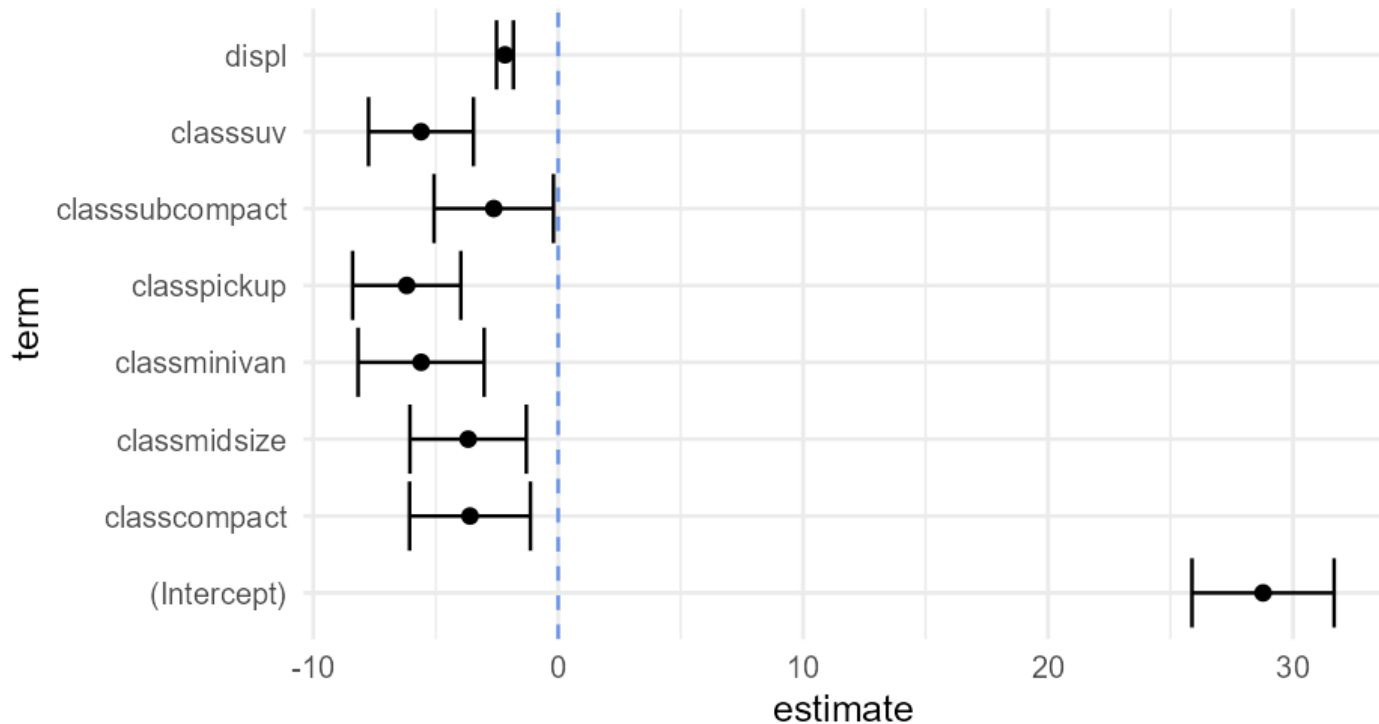
```
tidied_m <- broom::tidy(m, conf.int = TRUE)
```

```
tidied_m
```

```
## # A tibble: 8 × 7
```

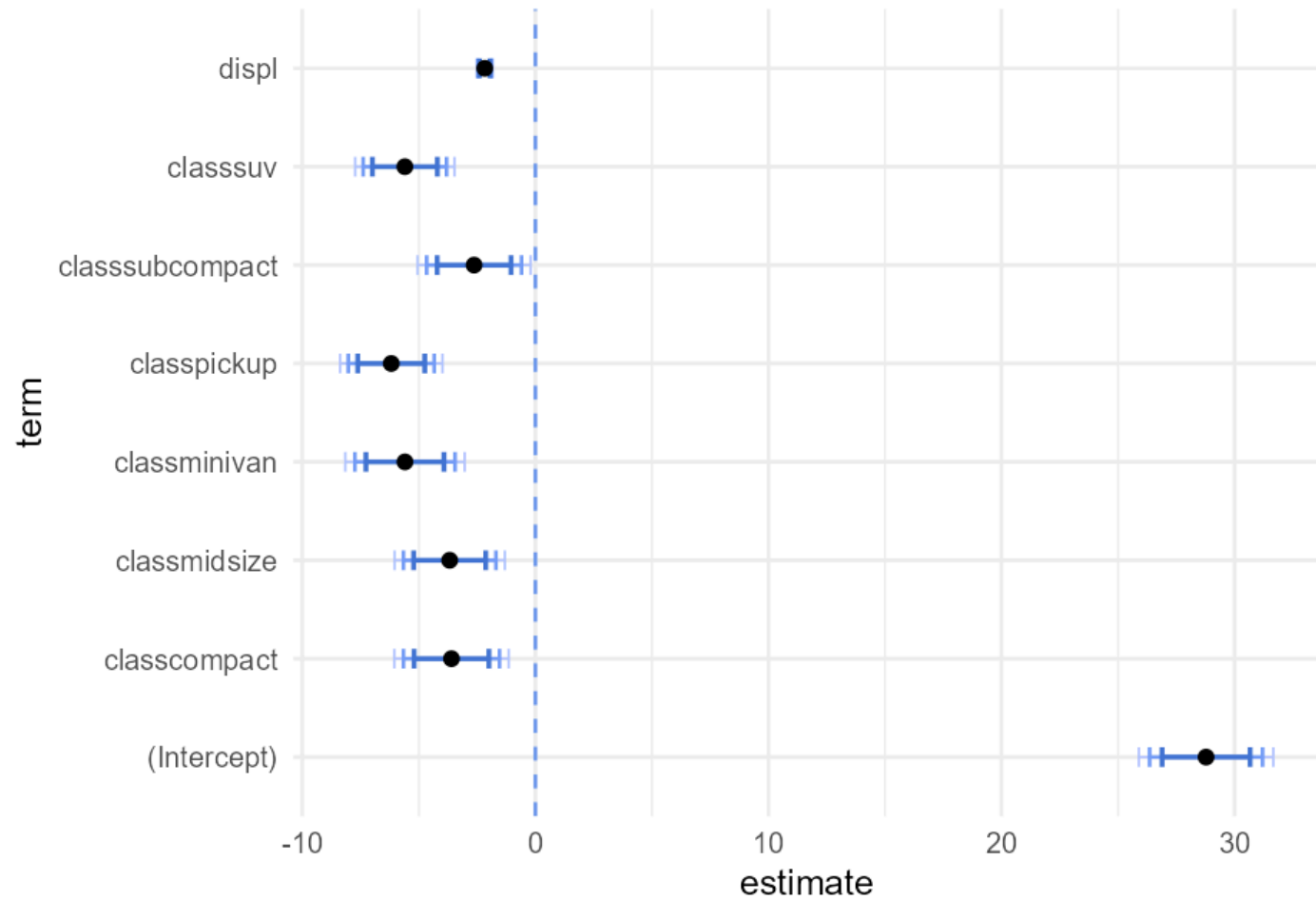
##	term	estimate	std.error	statistic	p.value	conf.low	conf.high
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	(Intercept)	28.8	1.47	19.5	1.91e-50	25.9	31.7
## 2	displ	-2.17	0.175	-12.4	2.20e-27	-2.52	-1.83
## 3	classcompact	-3.60	1.25	-2.87	4.44e- 3	-6.07	-1.13
## 4	classmidsize	-3.68	1.21	-3.05	2.59e- 3	-6.05	-1.30
## 5	classminivan	-5.60	1.31	-4.28	2.71e- 5	-8.17	-3.02
## 6	classpickup	-6.18	1.12	-5.51	9.60e- 8	-8.39	-3.97
## 7	classsubcompact	-2.63	1.24	-2.13	3.46e- 2	-5.07	-0.192
## 8	classsuv	-5.60	1.09	-5.15	5.65e- 7	-7.74	-3.46


```
ggplot(tidied_m, aes(term, estimate)) +
  geom_hline(yintercept = 0,
             color = "cornflowerblue",
             linetype = 2) +
  geom_errorbar(aes(ymin = conf.low, ymax = conf.high)) +
  geom_point() +
  coord_flip()
```



Multiple error bars

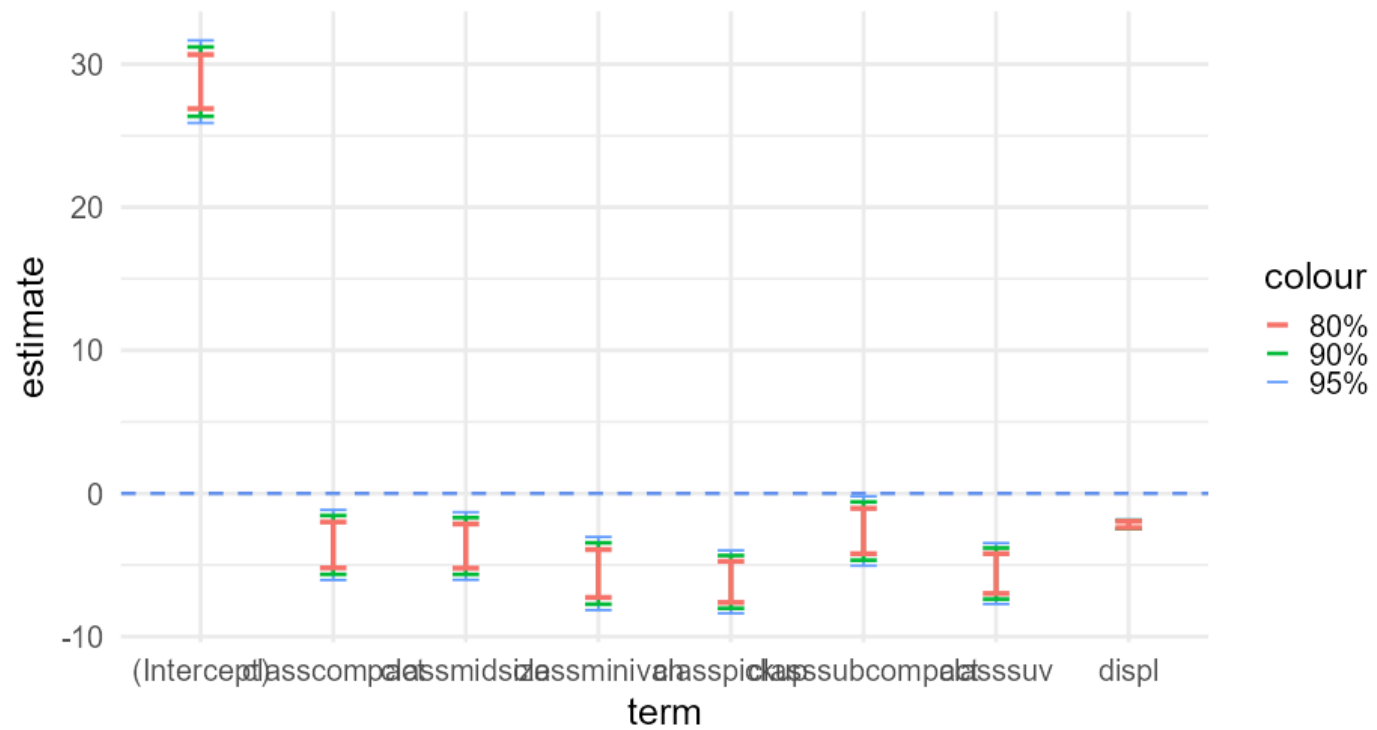
```
library(colorspace)
ggplot(tidied_m, aes(term, estimate)) +
  geom_hline(yintercept = 0,
             color = "cornflowerblue",
             linetype = 2) +
  geom_errorbar(aes(ymin = estimate + qnorm(.025)*std.error,
                    ymax = estimate + qnorm(.975)*std.error),
               color = lighten("#4375D3", .6),
               width = 0.2,
               size = 0.8) + # 95% CI
  geom_errorbar(aes(ymin = estimate + qnorm(.05)*std.error,
                    ymax = estimate + qnorm(.95)*std.error),
               color = lighten("#4375D3", .3),
               width = 0.2,
               size = 1.2) + # 90% CI
  geom_errorbar(aes(ymin = estimate + qnorm(.1)*std.error,
                    ymax = estimate + qnorm(.9)*std.error),
               color = "#4375D3",
               width = 0.2,
               size = 1.6) + # 80% CI
  geom_point() +
  coord_flip()
```



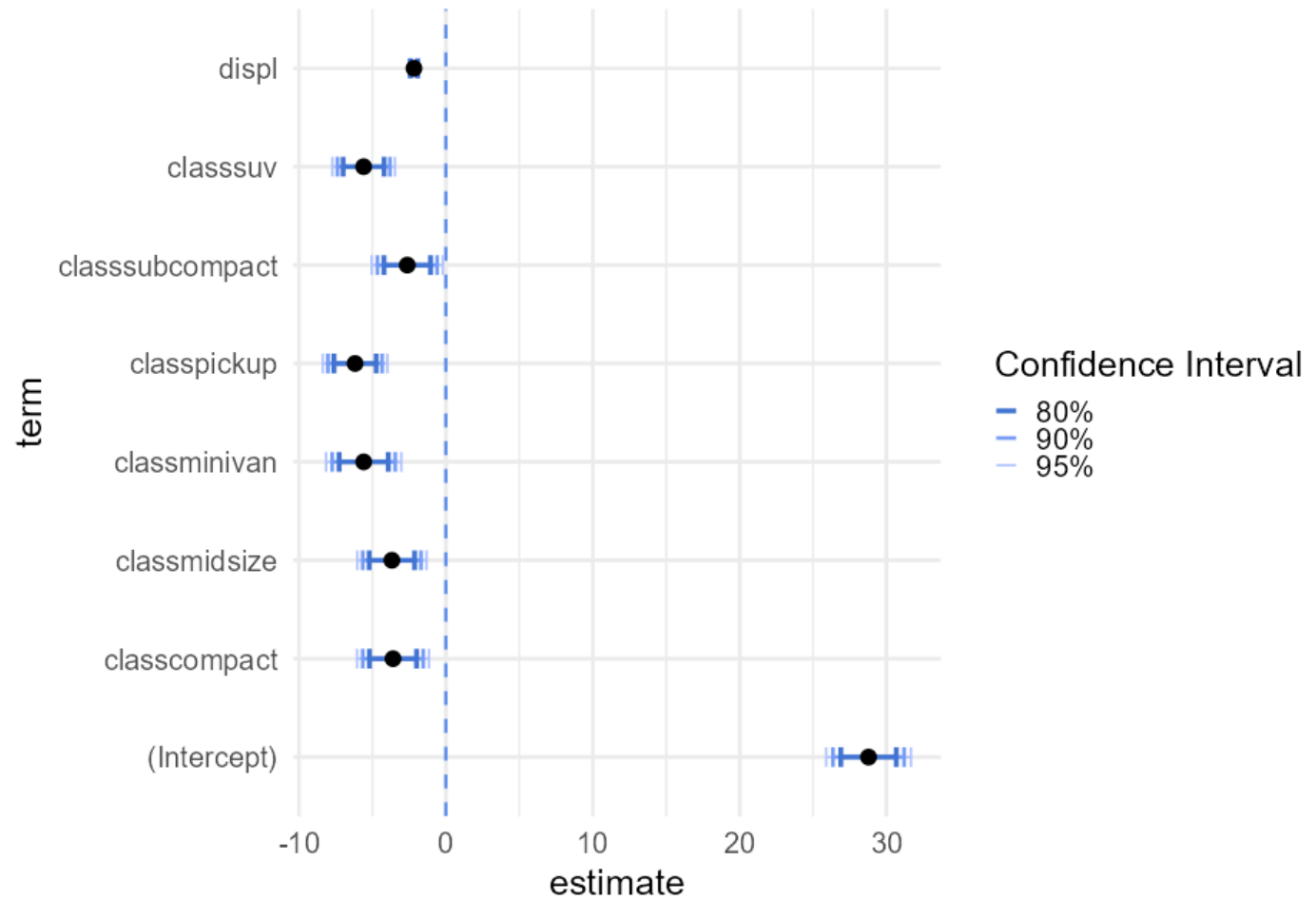
Add levels to legend

```
p <- ggplot(tidied_m, aes(term, estimate)) +  
  geom_hline(yintercept = 0,  
             color = "cornflowerblue",  
             linetype = 2) +  
  geom_errorbar(aes(ymin = estimate + qnorm(.025)*std.error,  
                   ymax = estimate + qnorm(.975)*std.error,  
                   color = "95%"),  
               width = 0.2,  
               size = 0.8) +  
  geom_errorbar(aes(ymin = estimate + qnorm(.05)*std.error,  
                   ymax = estimate + qnorm(.95)*std.error,  
                   color = "90%"),  
               width = 0.2,  
               size = 1.2) +  
  geom_errorbar(aes(ymin = estimate + qnorm(.1)*std.error,  
                   ymax = estimate + qnorm(.9)*std.error,  
                   color = "80%"),  
               width = 0.2,  
               size = 1.6)
```

p

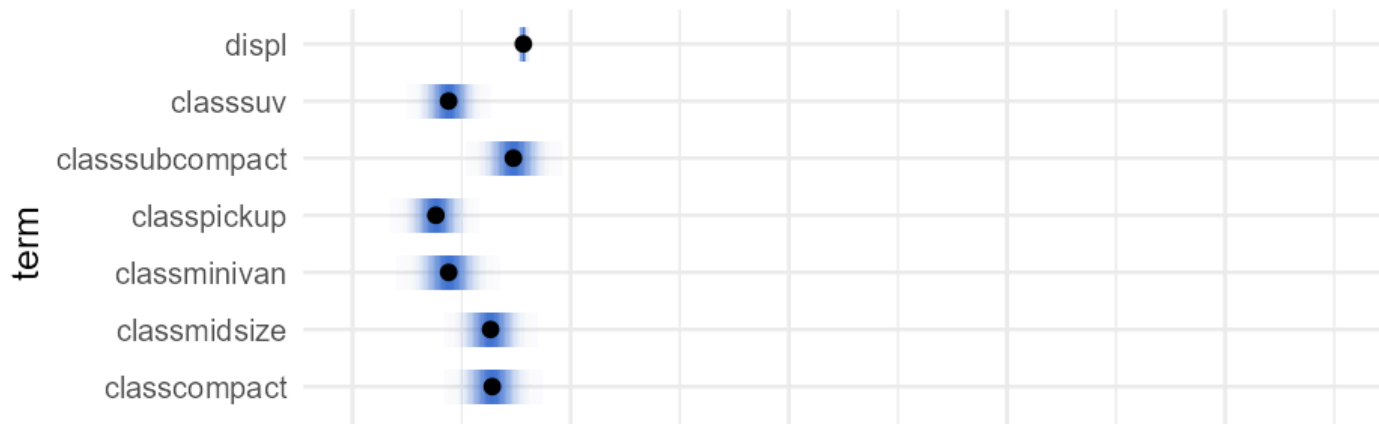


```
p +  
  scale_color_manual("Confidence Interval",  
                      values = c("#4375D3",  
                                lighten("#4375D3", .3),  
                                lighten("#4375D3", .6))) +  
  geom_point() +  
  coord_flip()
```



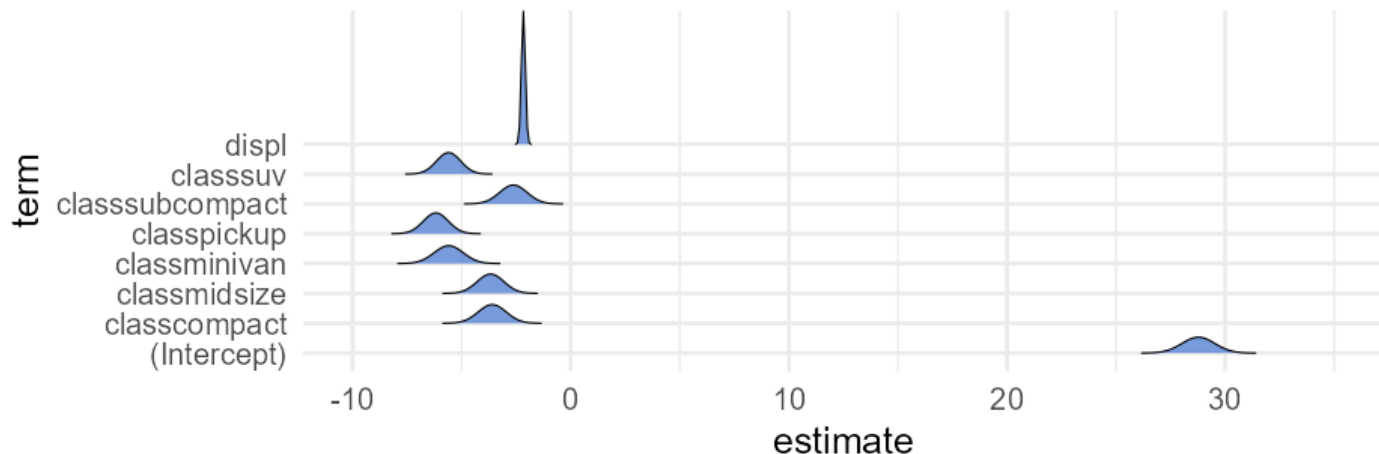
Density stripes

```
if (!requireNamespace("ungeviz", quietly = TRUE)) {  
  remotes::install_github("wilkelab/ungeviz")  
}  
#remotes::install_github("wilkelab/ungeviz")  
library(ungeviz)  
ggplot(tidied_m, aes(estimate, term)) +  
  stat_confidence_density(  
    aes(moe = std.error),  
    fill = "#4375D3",  
    height = 0.6  
  ) +  
  xlim(-10, 35) +  
  geom_point()
```



Actual densities

```
library(ggribes)
ggplot(tidied_m, aes(estimate, term)) +
  stat_confidence_density(
    aes(moe = std.error, height = stat(density)),
    geom = "ridgeline",
    confidence = 0.95,
    min_height = 0.001,
    alpha = 0.7,
    fill = "#4375D3"
  ) +
  xlim(-10, 35)
```

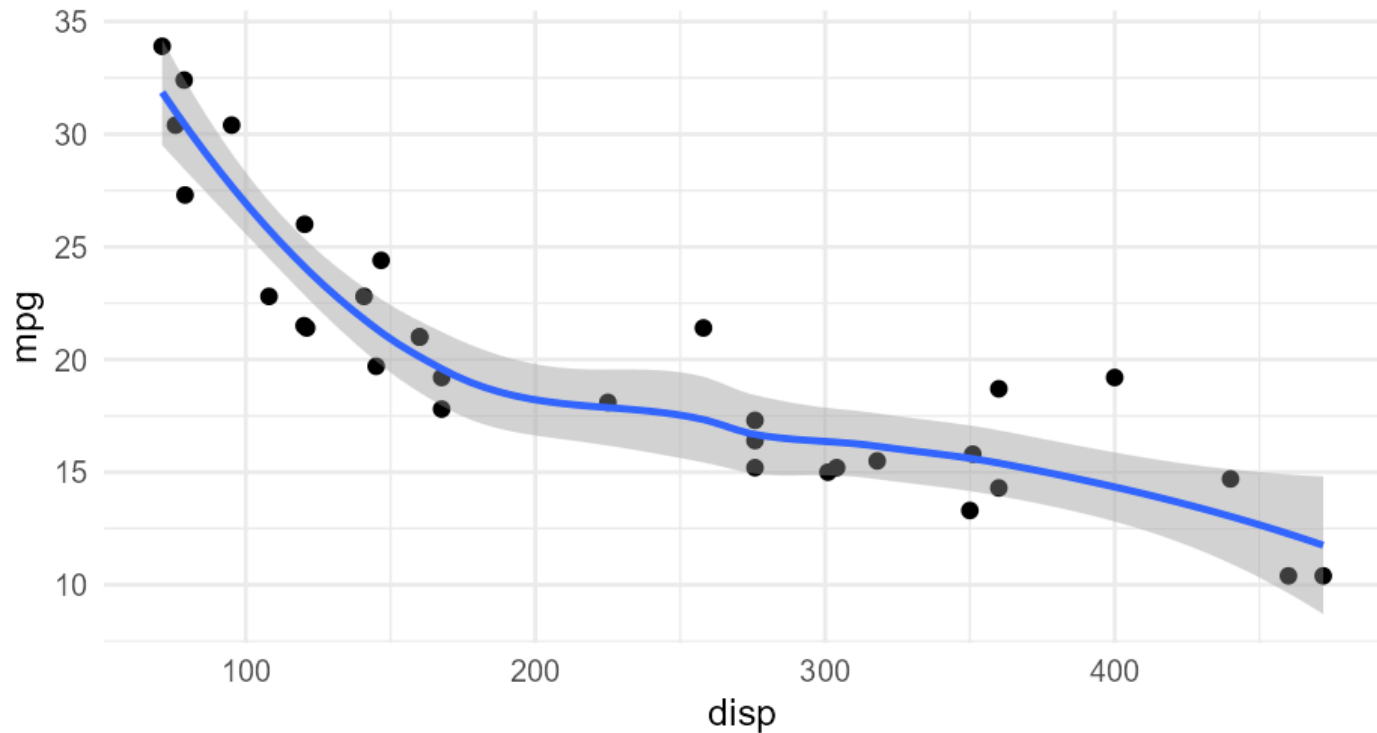


HOPs

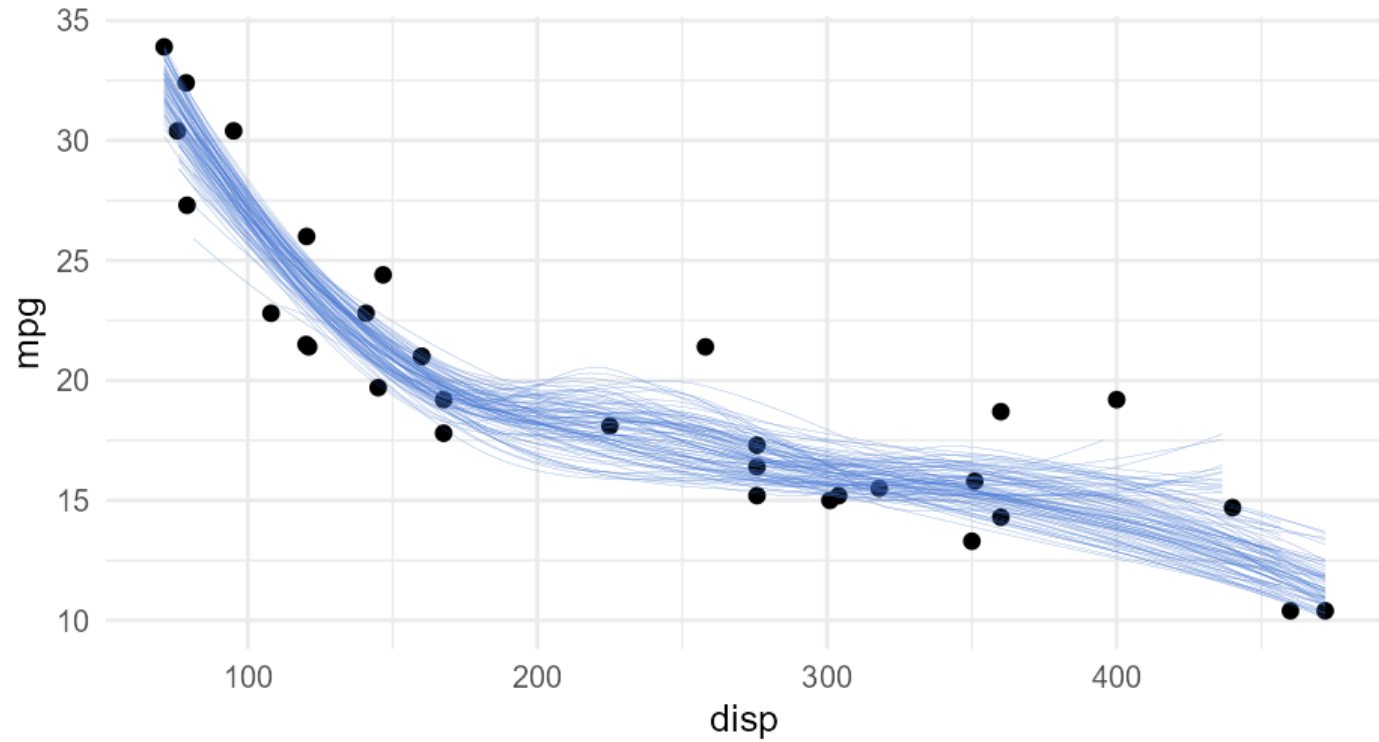
Hypothetical Outcome Plots (and
related plots)

Standard regression plot

```
ggplot(mtcars, aes(displacement, mpg)) +  
  geom_point() +  
  geom_smooth()
```



Alternative



How?

Bootstrapping

```
row_samps <- replicate(  
  100,  
  sample(  
    seq_len(nrow(mtcars)),  
    nrow(mtcars),  
    replace = TRUE  
  ),  
  simplify = FALSE  
)  
  
row_samps
```

```
## [[1]]  
## [1] 31 6 32 12 1 14 2 11 20 10 26 10 22 30 25 25 5 31 19 13 19 20 17 1 4  
##  
## [[2]]  
## [1] 11 23 14 1 24 20 10 30 27 24 22 23 25 1 18 18 25 8 8 16 25 19 31 13 11  
##  
## [[3]]  
## [1] 27 29 22 5 6 8 14 16 7 13 17 13 21 10 7 21 7 20 30 30 5 10 9 8 4  
##  
## [[4]]
```

Extract samples

```
d_samps <- map_df(row_samps, ~mtcars[.x, ], .id = "sample")
head(d_samps)
```

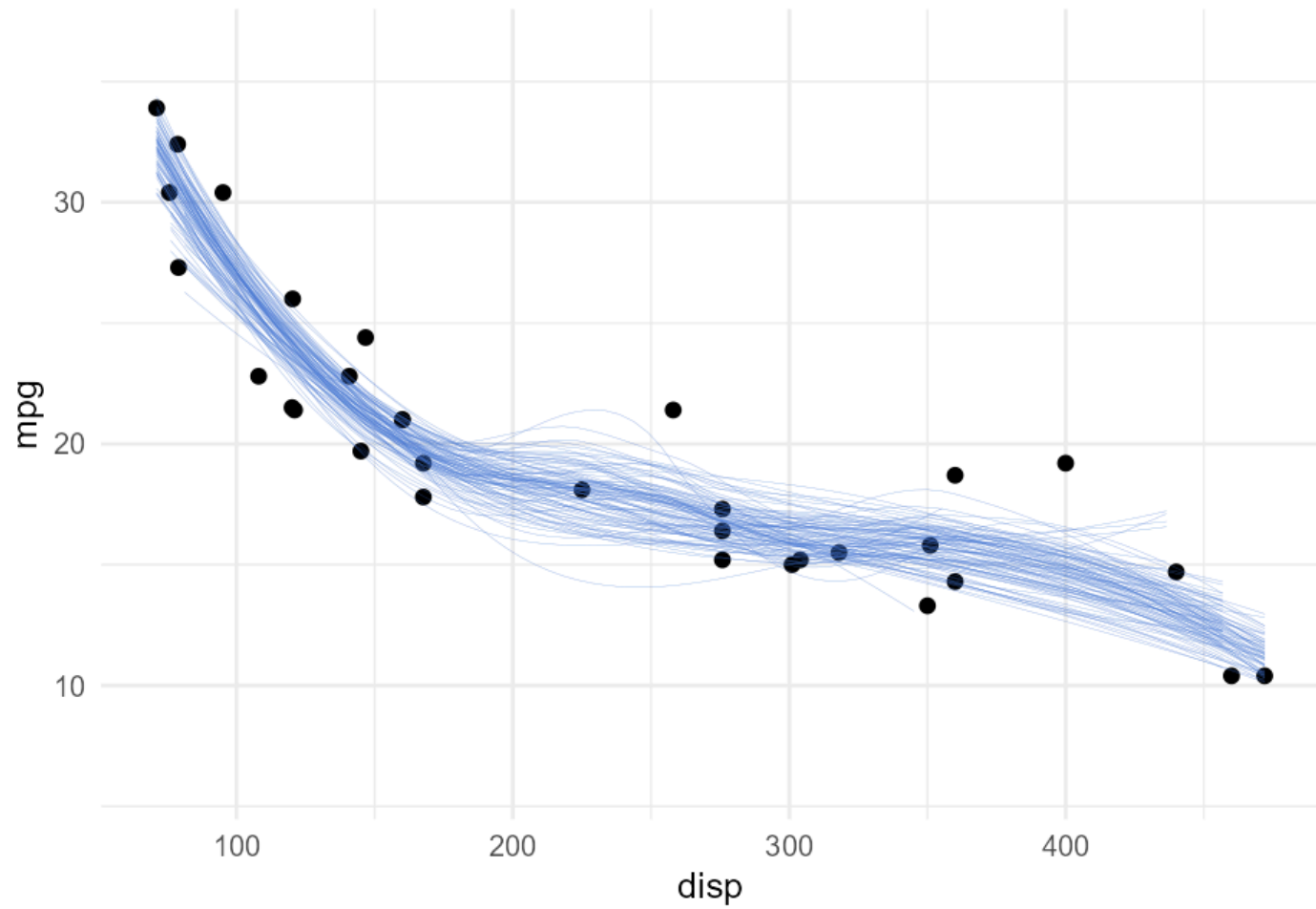
##	sample	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
## Maserati Bora...1	1	15.0	8	301.0	335	3.54	3.57	14.60	0	1	5	8
## Valiant...2	1	18.1	6	225.0	105	2.76	3.46	20.22	1	0	3	1
## Volvo 142E...3	1	21.4	4	121.0	109	4.11	2.78	18.60	1	1	4	2
## Merc 450SE...4	1	16.4	8	275.8	180	3.07	4.07	17.40	0	0	3	3
## Mazda RX4...5	1	21.0	6	160.0	110	3.90	2.62	16.46	0	1	4	4
## Merc 450SLC...6	1	15.2	8	275.8	180	3.07	3.78	18.00	0	0	3	3

```
tail(d_samps)
```

##	sample	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
## Lincoln Continental.1...3195	100	10.4	8	460.0	215	3.00	5.424	17.82	0	0		
## Pontiac Firebird...3196	100	19.2	8	400.0	175	3.08	3.845	17.05	0	0		
## Fiat X1-9.1...3197	100	27.3	4	79.0	66	4.08	1.935	18.90	1	1		
## Mazda RX4 Wag.3...3198	100	21.0	6	160.0	110	3.90	2.875	17.02	0	1		
## Hornet Sportabout...3199	100	18.7	8	360.0	175	3.15	3.440	17.02	0	0		
## Lotus Europa...3200	100	30.4	4	95.1	113	3.77	1.513	16.90	1	1		

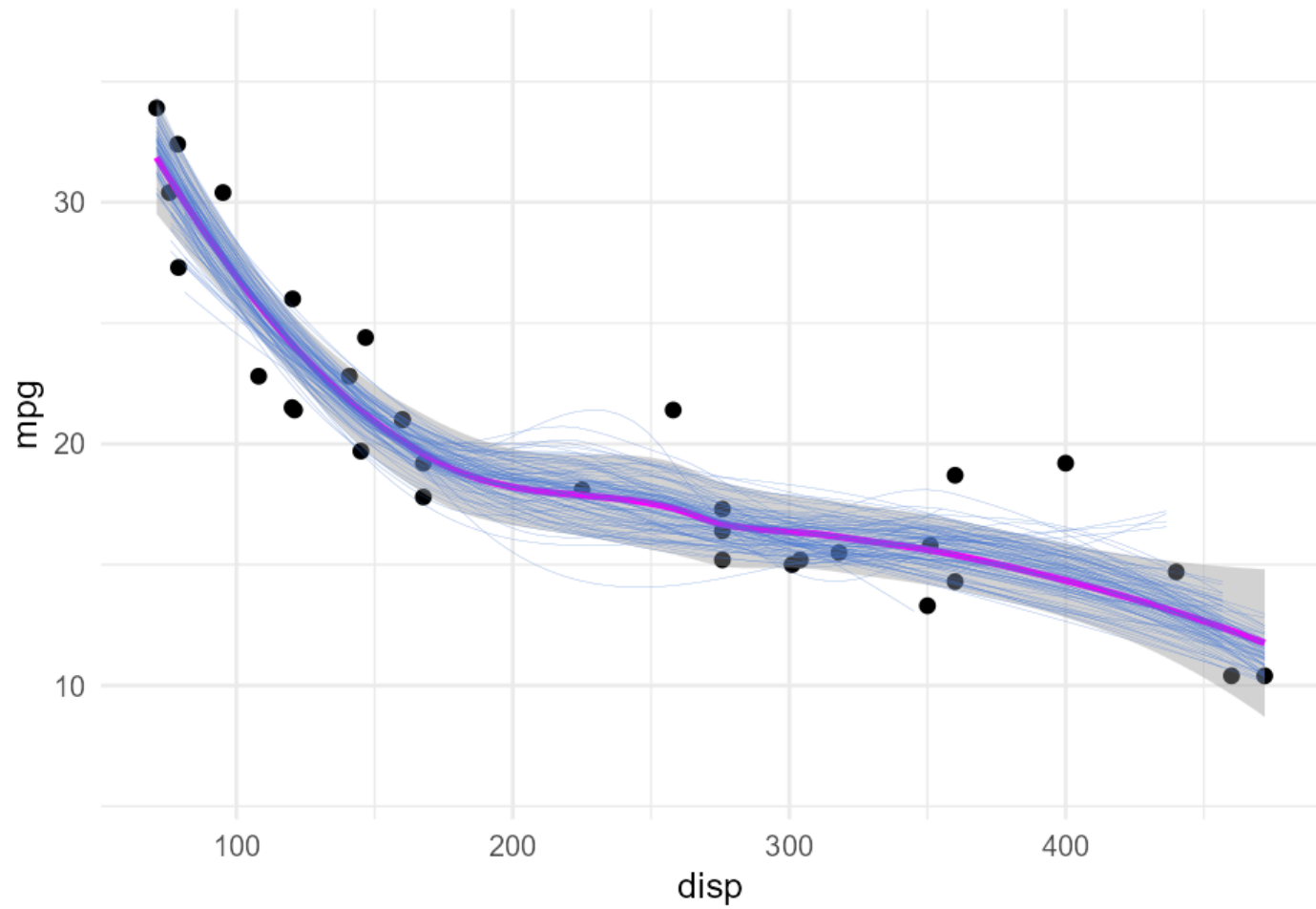
Plot both data sources

```
ggplot(mtcars, aes(displacement, mpg)) +  
  geom_point() +  
  stat_smooth(  
    aes(group = sample),  
    data = d_samps,  
    geom = "line",  
    color = "#4375D3",  
    fullrange = TRUE,  
    size = 0.1  
  )
```



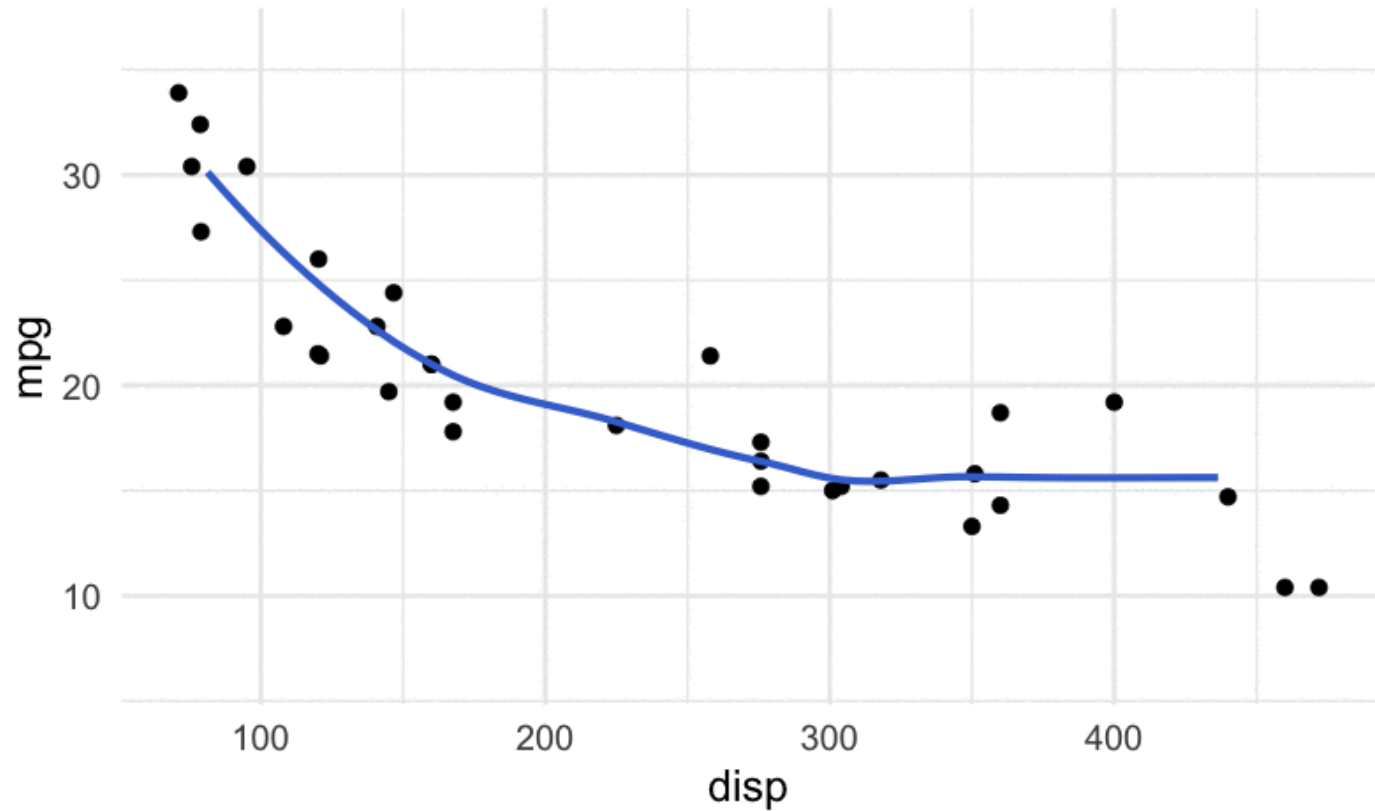
Note, they match up

```
ggplot(mtcars, aes(displacement, mpg)) +  
  geom_point() +  
  geom_smooth(color = "magenta") +  
  stat_smooth(  
    aes(group = sample),  
    data = d_samps,  
    geom = "line",  
    color = "#4375D3",  
    fullrange = TRUE,  
    size = 0.1  
  )
```



HOPs

Hops animate the process, so you can't settle on one "truth"

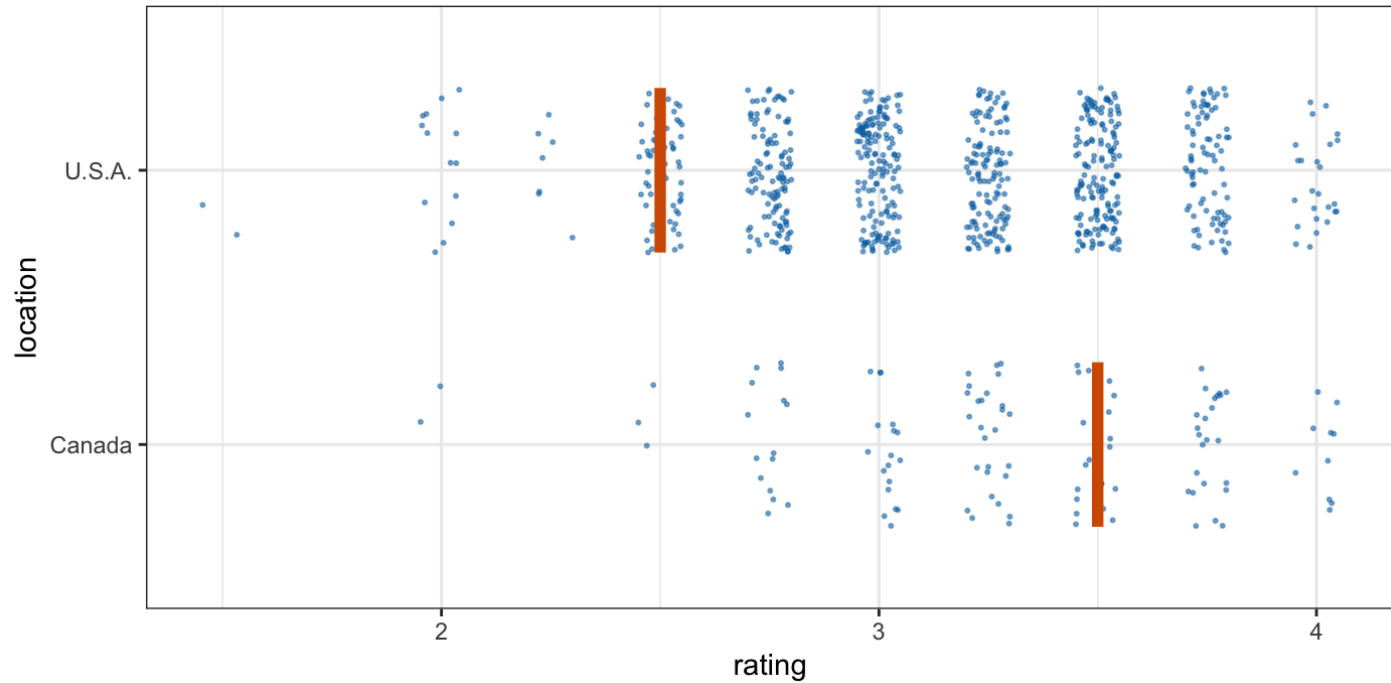


How?

gganimate::transition_states

```
library(gganimate)
ggplot(mtcars, aes(displ, mpg)) +
  geom_point() +
  stat_smooth(
    data = d_samps,
    geom = "line",
    size = 2,
    color = "#4375D3",
    fullrange = TRUE
  ) +
  transition_states(
    sample,
    transition_length = 0.5,
    state_length = 0.5
  ) +
  ease_aes('linear') # Smoother transitions
```

Another example



Another examples

From Dr. Kay again

Matthew Kay Keynote at Tapestry 2018: A biased tour of th...



Conclusions

- Lots of tools at your disposal (perhaps so many it can be difficult to choose)
- Do try to communicate uncertainty whenever possible
- I'd recommend checking out [Claus Wilke's talk](#) from [rstudio::conf\(2019L\)](#), where he talks about the [ungeviz](#) package.

Survey Weights in ggplot2

Why weights matter

- Survey data is almost never a simple random sample
- Some groups are **oversampled** (e.g., small racial/ethnic groups for precision)
- Others are **underrepresented** due to nonresponse
- Ignoring weights gives you **descriptive statistics that don't represent the population**

Two questions to ask

1. What does my *unweighted* plot actually represent?
2. Does my visualization claim to describe the population — or just my sample?

A toy example

```
library(tidyverse)

# Simulate a survey: pop is 80% Group A, 20% Group B
# But our sample overrepresents Group B (50/50)
set.seed(42)
survey_df <- tibble(
  group      = rep(c("A", "B"), each = 500),
  outcome    = c(rnorm(500, mean = 60, sd = 10),      # Group A
                 rnorm(500, mean = 75, sd = 10)),      # Group B
  weight     = rep(c(0.8 / 0.5, 0.2 / 0.5), each = 500)
  # weight = true pop share / sample share
)

head(survey_df)
```

```
## # A tibble: 6 × 3
##   group outcome weight
##   <chr>   <dbl>   <dbl>
## 1 A       73.7     1.6
## 2 A       54.4     1.6
## 3 A       63.6     1.6
## 4 A       66.3     1.6
## 5 A       64.0     1.6
## 6 A       58.9     1.6
```

Unweighted vs. weighted mean

```
survey_df %>%  
  summarize(  
    unweighted_mean = mean(outcome),  
    weighted_mean   = weighted.mean(outcome, w = weight)  
  )
```

```
## # A tibble: 1 × 2  
##   unweighted_mean weighted_mean  
##           <dbl>         <dbl>  
## 1           67.2           62.7
```

The true population mean (80% A at 60, 20% B at 75) should be:

$$0.8 * 60 + 0.2 * 75$$

```
## [1] 63
```

Unweighted inflates Group B's contribution.
Weighted recovers the population value.

Option 1: Pre-compute weighted stats

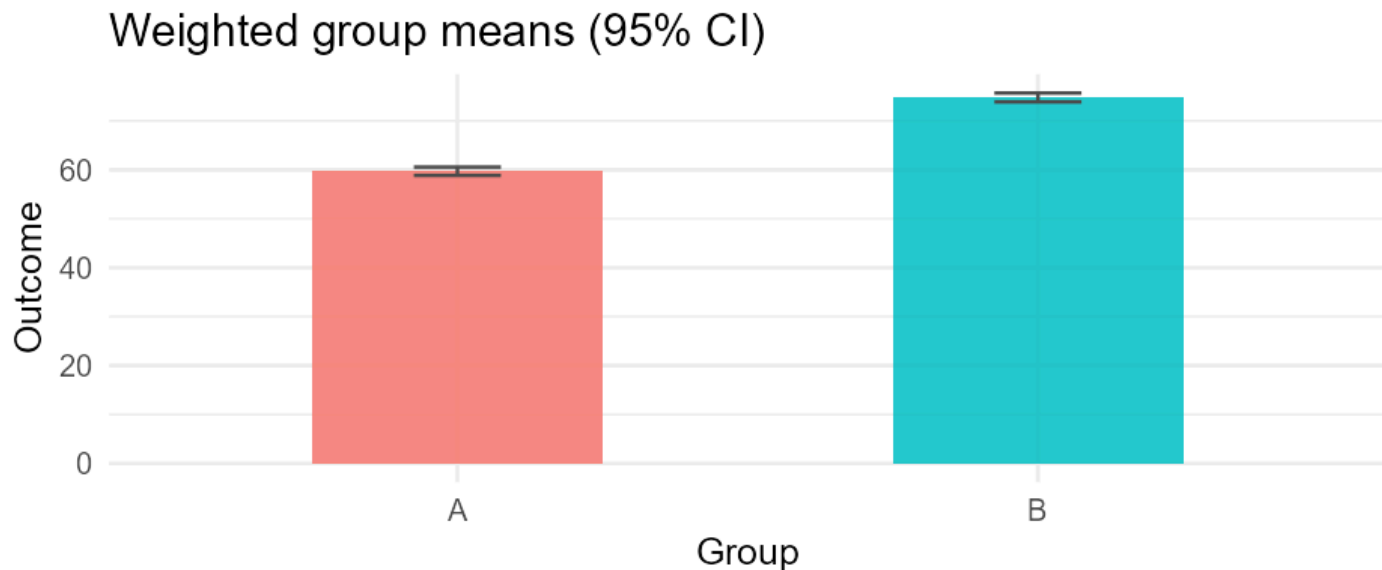
The most transparent approach — calculate weighted summaries **before** plotting.

```
weighted_stats <- survey_df %>%  
  group_by(group) %>%  
  dplyr::summarize(  
    wmean = sum(outcome * weight) / sum(weight),  
    wse    = sqrt(sum(weight * (outcome - wmean)^2) / sum(weight)  
    .groups = "drop"  
  )
```

```

ggplot(weighted_stats, aes(group, wmean, fill = group)) +
  geom_col(width = 0.5, alpha = 0.85) +
  geom_errorbar(
    aes(ymin = wmean - 1.96 * wse,
        ymax = wmean + 1.96 * wse),
    width = 0.15,
    color = "gray30"
  ) +
  labs(
    title = "Weighted group means (95% CI)",
    x = "Group", y = "Outcome"
  ) +
  guides(fill = "none")

```



Option 2: Use the `{survey}` package

`{survey}` handles complex designs — stratification, clustering, finite-population corrections.

```
install.packages("survey")
```

```
library(survey)
svy_design <- svydesign(
  ids      = ~1,          # no clustering (PSU = individual)
  weights  = ~weight,
  data     = survey_df
)
```

```
svymean(~outcome, svy_design)
```

```
##           mean      SE  
## outcome 62.716 0.3908
```

```
svyby(~outcome, ~group, svy_design, svymean)
```

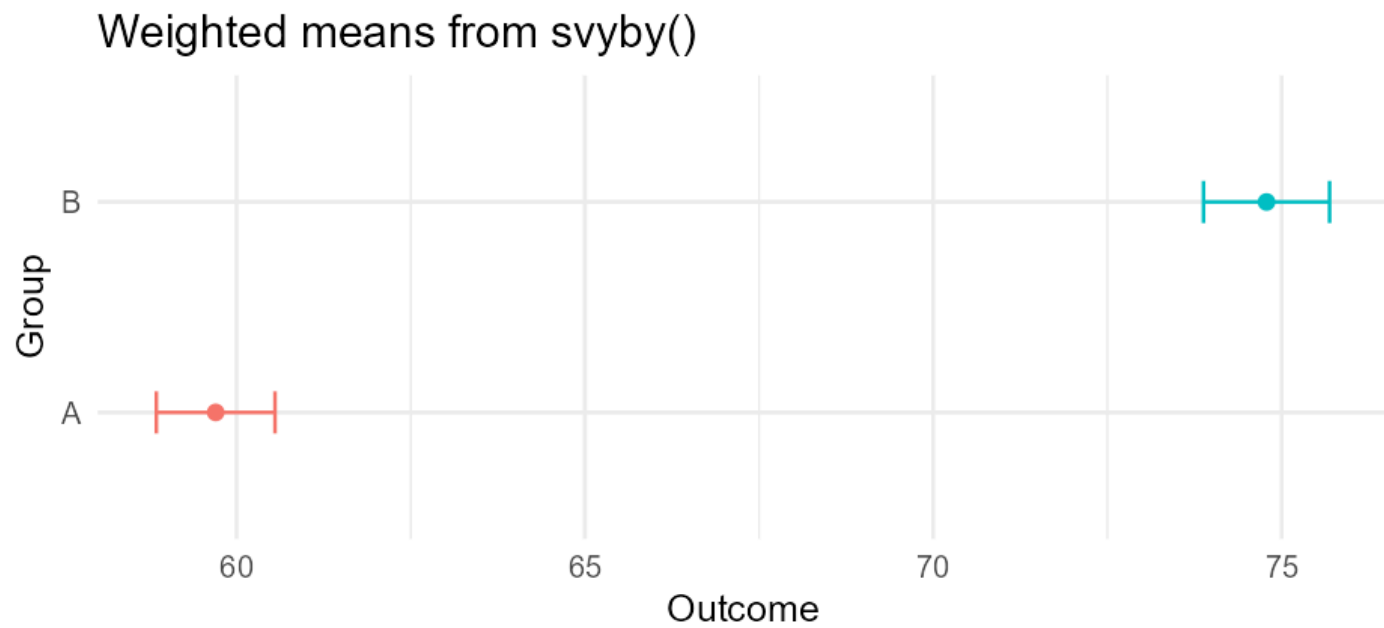
```
##  group outcome      se  
## A      A 59.69954 0.4345537  
## B      B 74.78397 0.4617093
```


Plotting `{survey}` output with `ggplot2`

`svyby()` returns a plain data frame — pipe it straight into `ggplot`.

```
svy_means <- svyby(~outcome, ~group, svy_design, svymean) %>%
  rename(wmean = outcome, wse = se)

ggplot(svy_means, aes(wmean, group, color = group)) +
  geom_errorbarh(
    aes(xmin = wmean - 1.96 * wse,
        xmax = wmean + 1.96 * wse),
    height = 0.2
  ) +
  geom_point(size = 4) +
  guides(color = "none") +
  labs(title = "Weighted means from svyby()", x = "Outcome", y =
```



Option 3: `aes(weight = )` in `ggplot2`

`ggplot` has a built-in `weight` aesthetic — but use it carefully.

```
ggplot(survey_df, aes(outcome, fill = group, weight = weight)) +  
  geom_density(alpha = 0.5) +  
  labs(title = "Weighted density curves")
```

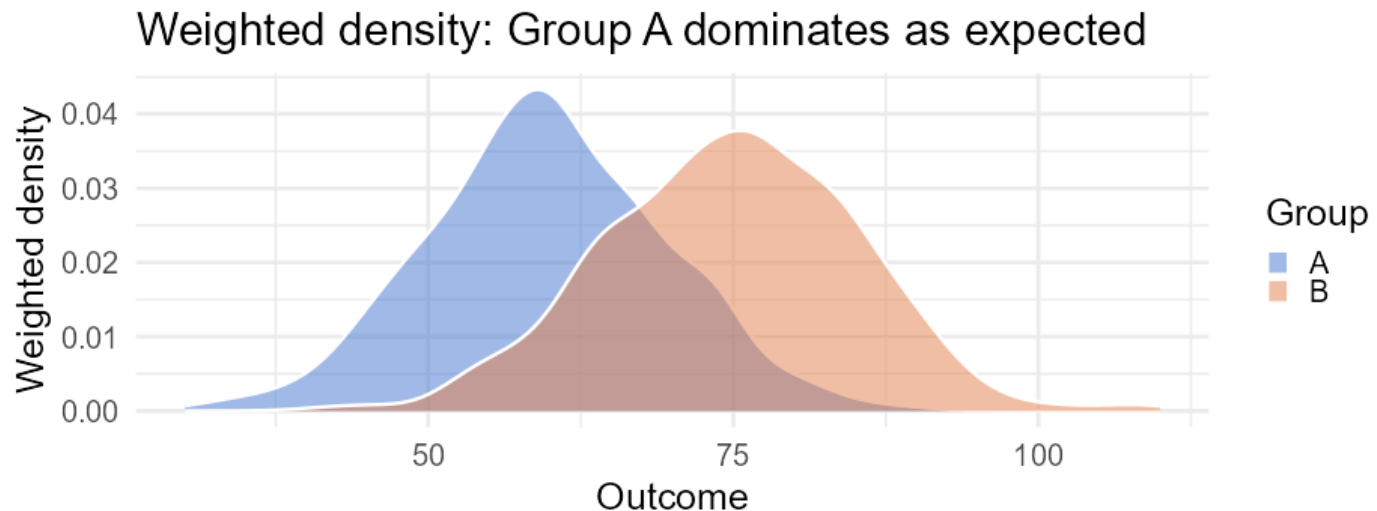
What it does and doesn't do

Geom	Respects weight =?	Notes
<code>geom_bar /</code> <code>geom_histogram</code>	✓ Yes	Sums weights per bin
<code>geom_density</code>	✓ Yes	Scales density by weight
<code>geom_boxplot</code>	✗ No	Ignores weights
<code>geom_smooth</code>	⚠ Partial	Only for some methods
<code>stat_summary</code>	✗ No	Must pre-compute

aes(weight =)

example

```
ggplot(survey_df, aes(outcome, fill = group, weight = weight)) +  
  geom_density(alpha = 0.5, color = "white") +  
  scale_fill_manual(values = c("#4375D3", "#E8834C")) +  
  labs(  
    title = "Weighted density: Group A dominates as expected",  
    x = "Outcome", y = "Weighted density", fill = "Group"  
  )
```

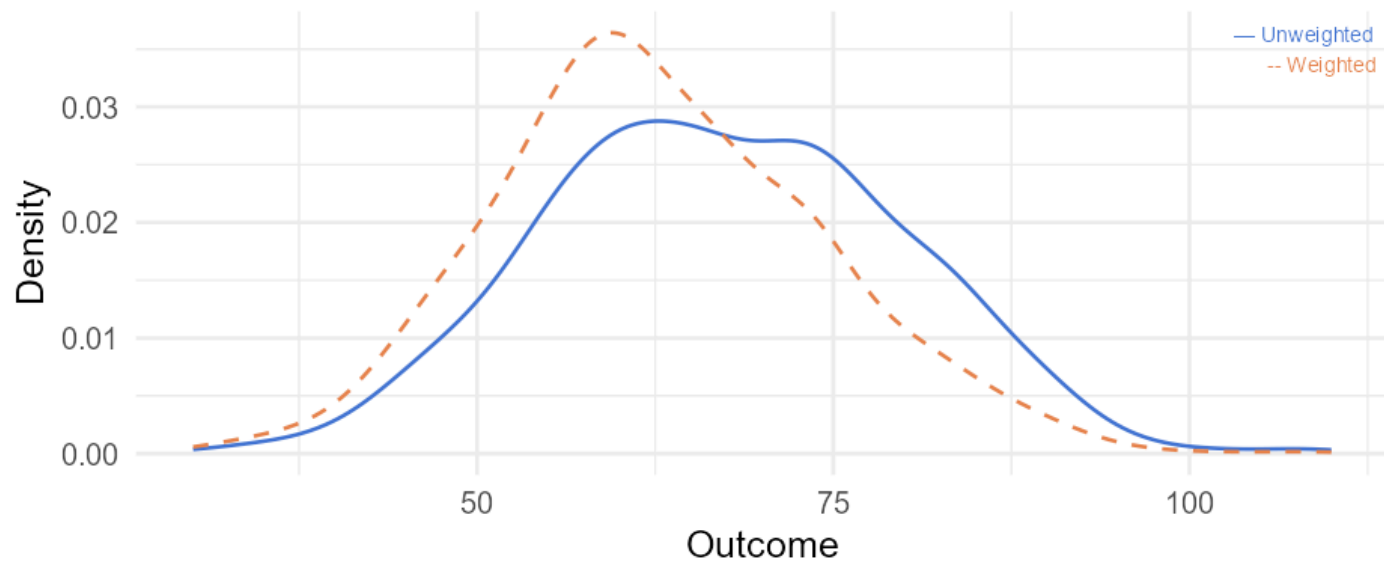


Unweighted vs. weighted density

```
unweighted_df <- survey_df
weighted_df    <- survey_df

ggplot(mapping = aes(x = outcome)) +
  geom_density(
    data = unweighted_df,
    color = "#4375D3",
    size = 1.2
  ) +
  geom_density(
    data = weighted_df,
    aes(weight = weight),
    color = "#E8834C",
    size = 1.2,
    linetype = 2
  ) +
  annotate("text", x = Inf, y = Inf,
    label = "-- Unweighted", color = "#4375D3",
    hjust = 1.1, vjust = 2, size = 5) +
  annotate("text", x = Inf, y = Inf,
    label = "-- Weighted", color = "#E8834C",
```

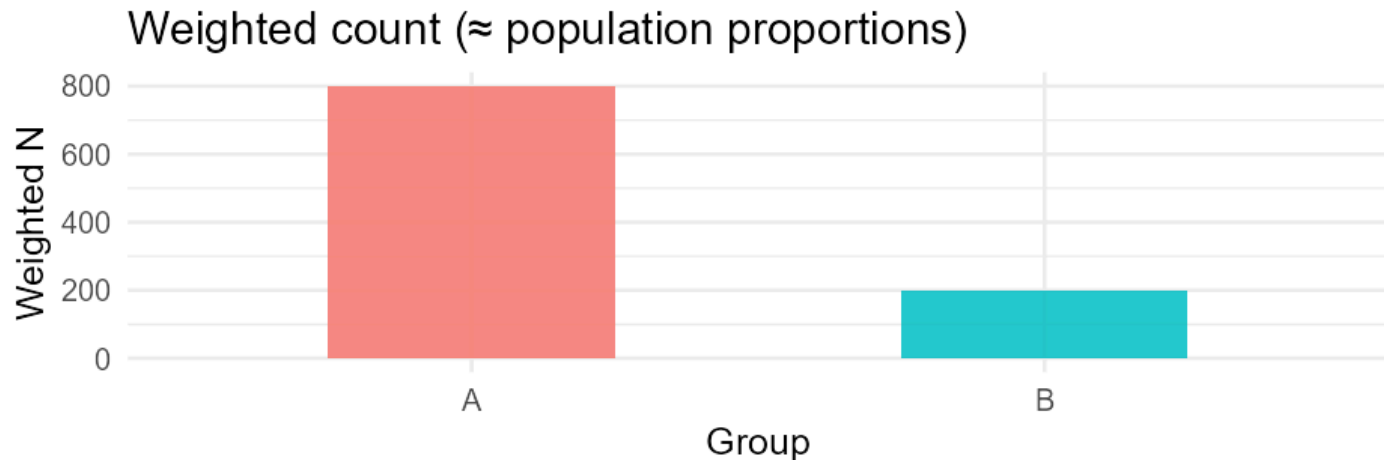
Unweighted vs. weighted density



Weighted bar charts

`geom_bar()` with `weight` sums weights rather than counting rows — giving **population estimates**.

```
ggplot(survey_df, aes(x = group, weight = weight)) +  
  geom_bar(aes(fill = group), alpha = 0.85, width = 0.5) +  
  guides(fill = "none") +  
  labs(  
    title = "Weighted count (≈ population proportions)",  
    x = "Group", y = "Weighted N"  
  )
```



Real-world workflow: YRBS

```
# Typical pattern with education survey data
weighted_summary <- your_survey_data %>%
  group_by(year, subgroup) %>%
  summarize(
    est  = wtd.mean(outcome_var, weights = analytic_weight),
    se   = sqrt(wtd.var(outcome_var, weights = analytic_weight)),
    low  = est - 1.96 * se,
    high = est + 1.96 * se,
    .groups = "drop"
  )

ggplot(weighted_summary, aes(year, est, color = subgroup)) +
  geom_line() +
  geom_ribbon(aes(ymin = low, ymax = high, fill = subgroup),
             alpha = 0.15, color = NA) +
  labs(title = "Weighted trend with 95% CI band")
```

Key takeaways: survey weights

- Always **pre-compute** weighted means/SEs for `stat_summary`-style geoms — they don't weight internally
- `aes(weight = )` works well for `geom_density`, `geom_bar`, and `geom_histogram` but silently ignores weights in others
- `{survey}` is the gold standard for complex designs; its output is ggplot-ready via `svyby()`

Key takeaways: survey weights

- Label your plots — readers should know whether estimates are weighted and what population they represent
- Mismatch between weighted and unweighted can itself be a **finding worth visualizing**

Shiny App Deployment

Why deploy a Shiny app?

- Static plots are great — but sometimes your audience needs to **explore** the data themselves
- Shiny lets users filter, subset, and interact without touching code
- Deployment makes your app accessible to anyone with a browser
- Common deployment targets:
 - **shinyapps.io** — easiest, hosted by Posit
 - **Posit Connect** — institutional/enterprise option
 - **Hugging Face Spaces** or **Render** — free-tier alternatives
 - **Quarto blog** — embed directly in a post (with caveats)

Deploying to shinyapps.io

One-time setup

```
install.packages("rsconnect")
library(rsconnect)

# Authenticate with your shinyapps.io token
rsconnect::setAccountInfo(
  name    = "your-username",
  token   = "YOUR_TOKEN",
  secret  = "YOUR_SECRET"
)
```

Get your token at: shinyapps.io → Account → Tokens

Deploy in one line

```
rsconnect::deployApp(  
  appDir = "path/to/your/app",    # folder containing app.R or ui.R  
  appName = "my-uncertainty-app"  # URL slug on shinyapps.io  
)
```

- `appDir` defaults to the working directory if omitted
- The free tier allows **5 apps** and **25 active hours/month**
- Re-deploy anytime by running the same command — it updates in place

App folder structure

```
my-app/  
├─ app.R          # Combined ui + server (preferred for simple apps)  
├─ data/  
  └─ mydata.csv
```

Or the split version:

```
my-app/  
├─ ui.R  
├─ server.R  
└─ global.R      # Shared objects loaded once
```

Tip: Keep data files small or read from a URL —
shinyapps.io has a 1 GB bundle limit.

Shiny apps in a Quarto blog

There are two main approaches:

Option 1: `<iframe>` embed (recommended)

Host your app on shinyapps.io, then embed it in any `.qmd` post:

```
<iframe
  src="https://your-username.shinyapps.io/my-app/"
  width="100%"
  height="600px"
  frameborder="0">
</iframe>
```

- ✓ Works with static Quarto blogs (e.g., Netlify deploy)

Option 2: shinylive (self-contained)

shinylive runs R entirely in the browser via WebAssembly
— no server required.

```
install.packages("shinylive")  
quarto add quarto-ext/shinylive # add the Quarto extension
```

Then in a `.qmd` post, use a `shinylive-r` code chunk:

```
```{shinylive-r}
#| standalone: true
library(shiny)
ui <- fluidPage(
 sliderInput("n", "Sample size", 10, 500, 100),
 plotOutput("hist")
)
server <- function(input, output) {
 output$hist <- renderPlot({
 hist(rnorm(input$n), main = paste("n =", input$n))
 })
}
shinyApp(ui, server)
```
```

shinylive trade-offs

| Feature | shinyapps.io +
iframe | shinylive |
|-------------------------------|--------------------------|-----------------------------------|
| Needs a server | ✓ Yes | ✗ No |
| Package support | Full CRAN | Limited (compiled pkgs may fail) |
| Load time | Fast | Slow (downloads R runtime ~20 MB) |
| Works on Netlify/GitHub Pages | ✓ Yes | ✓ Yes |
| Best for | Complex apps | Simple demos |

For data viz courses and public-facing posts, **iframe embedding** is usually the most reliable approach.

Workflow for embedding in your blog

1. Build and test your app locally
2. Deploy to shinyapps.io with `rsconnect::deployApp()`
3. In your Quarto blog post (`.qmd`), paste the `<iframe>` tag
4. Render and push — the blog stays static, the app lives on shinyapps.io

```
# In your post's YAML front matter, no special options needed:  
title: "Uncertainty Visualizer"  
format: html
```

A minimal working example

```
# app.R – deploy this to shinyapps.io
library(shiny)
library(ggplot2)

ui <- fluidPage(
  titlePanel("Sampling Distribution Explorer"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("n", "Sample size", min = 5, max = 500,
        value = 100),
      sliderInput("reps", "Repetitions", min = 100, max = 5000,
        value = 1000),
      sliderInput("sd", "Population SD", min = 1, max = 30,
        value = 10),
    ),
    mainPanel(plotOutput("dist_plot"))
  )
)

server <- function(input, output) {
  output$dist_plot <- renderPlot({
    means <- replicate(input$reps, mean(rnorm(input$n, 100, input$sd)))
    ggplot(data.frame(x = means), aes(x)) +
      geom_histogram(fill = "#4375D3", color = "white", bins = 40) +
      labs(title = paste("SE = ", round(input$sd / sqrt(input$reps), 2)))
  })
}
```

Key takeaways for Shiny deployment

- **shinyapps.io** is the path of least resistance for most course projects
- **<iframe> embedding** is the most compatible way to include live apps in a Quarto blog
- **shinylive** is powerful for simple, self-contained demos but has real limitations for heavier packages
- Always test locally with **shiny::runApp()** before deploying
- Document your app's dependencies with **renv** or a clear **library()** list at the top of **app.R**

Data Viz in the wild

- Kyla and Isha

Final
presentations
from 3
groups
